Universidade da Beira Interior
Departamento de Informática

Evaluating the Performance of Java Application Caches in Web and Database-Driven Applications

Dissertação

Elaborado por:

Vasco Moreira Senra

Orientadores:

Professor Doutor Paul Crocker

Professor Doutor Valderi Leithardt

Agradecimentos

Gostaria de expressar a minha profunda gratidão a todos aqueles que contribuíram para a realização desta dissertação intitulada *"Evaluating the Performance of Java Application Caches in Web and Database-Driven Applications."*

Em primeiro lugar, agradeço sinceramente aos meus orientadores, Professor Doutor Paul Crocker e Valderi Leithardt pelas suas orientações, incentivos e apoios constantes ao longo de todo o processo. As suas ideias e sugestões foram essenciais para o desenvolvimento e sucesso deste trabalho.

Agradeço também aos professores e colegas de curso que me inspiraram e apoiaram durante a minha trajetória académica. As suas contribuições e críticas construtivas foram indispensáveis para o meu crescimento pessoal e profissional.

Finalmente, deixo uma palavra especial de agradecimento à minha família e amigos, que estiveram sempre presentes com amor, incentivo e apoio em cada etapa da minha jornada. O apoio incondicional foi crucial para me ajudar a superar desafios e a alcançar este importante objetivo.

A todos, a minha sincera gratidão e apreço.

Conteúdo

Conteúdo **ii**

Lista de Figuras **vi**

1 Introdução **1**
1.1 Enquadramento 1
1.2 Motivação 1
1.3 Objetivos 2
1.4 Organização do Documento 3

2 Conceitos Fundamentais de *Cache* e Aplicações Web **5**
2.1 Introdução 5
2.2 *Cache* 5
2.2.1 Definição 5
2.2.2 Princípios de Funcionamento 6
2.2.3 Benefícios e Limitações 7
2.3 Tipos Clássicos de Armazenamento em *Cache* 7
2.3.1 Aplicações e Usos da Cache 8
2.4 Arquiteturas de Desenvolvimento *Web* 8
2.4.1 Arquitetura em Duas Camadas 8
2.4.2 Arquitetura em Três Camadas 9
2.5 Modelos de *Cache* em Aplicações Web 9
2.5.1 *Cache* em Três Camadas 9
2.5.2 *Cache* em Duas Camadas 10
2.5.3 Boas Práticas de Implementação 10
2.5.4 Coerência dos Dados 11
2.5.5 Caso de Estudo:EPOS 11
2.6 Conclusões 13

3 Estado da Arte **15**
3.1 Introdução 15
3.2 Método de Pesquisa 15
3.2.1 Strings de Pesquisa 15

3.2.2 Repositórios de Pesquisa 16
3.2.3 Critérios de Seleção 17
3.2.4 Critérios de Inclusão 17
3.2.5 Síntese dos Resultados 17
3.3 Revisão dos estudos mais importantes 17
3.3.1 A Performance Comparison of Java Cache Memory Implementations 17
3.3.2 An Overview of Analysis Methods and Evaluation Results for Caching Strategies 18
3.3.3 JVM Optimization: An Empirical Analysis of JVM Configurations for Enhanced Web Application Performance . 18
3.3.4 Redis vs. Memcached in Microservices Architectures: Caching Strategies 19
3.3.5 Database Loading Strategies for an In-Memory Cache in Java 19
3.3.6 Caching Strategy for Web Application – A Systematic Literature Review 20
3.3.7 Conclusão 20
3.4 Bibliotecas de *cache* para *Java* 21
3.4.1 *Guava* 21
3.4.2 *Apache Ignite* 22
3.4.3 *Ehcache* 22
3.4.4 *Cache4j* 22
3.4.5 *Caffeine* 22
3.4.6 *Payara* 22
3.4.7 *Redis* 23
3.4.8 Escolhas para o estudo 23
3.4.9 Conclusão 24

4 Metodologia 25
4.1 Introdução 25
4.2 Ambiente de Execução 25
4.2.1 Grupo de Controlo 25
4.3 Definição das Métricas 26
4.3.1 Concorrência 26
4.3.2 Implementação dos Mecanismos de *Cache* 27
4.4 Criação dos Testes 27
4.5 Análise dos Dados 27
4.5.1 Conclusão 28

5 Arquitetura do Sistema 29

5.1 Lógica do Serviço 29
5.2 Especificações de Hardwares 30
5.3 Questões técnicas 32
5.4 Divisão do Sistema 33
5.4.1 Backend 33
5.4.1.1 Esquema da Base de Dados 33
5.4.1.2 Geração de Dados 36
5.4.1.3 Desafios 37
5.4.2 API - Base 38
5.4.2.1 Lógica 38
5.4.2.2 Workflow 38
5.5 Requests 40
5.5.1 Endpoints 40
5.6 Parser 41
5.7 Parâmetros 43
5.8 Coleta de Dados 43
5.8.1 Erros 44
5.8.2 Limitações de String 44
5.8.3 Heap Size (8 GB) 44
5.9 Chamadas à API 44
5.9.1 Definição dos Endpoints 45
5.9.2 Geração de Parâmetros 45
5.9.3 Construção da URL Final 46
5.9.4 Funções de Geração de Dados 47
5.10 Avaliação da API : Concorrência 47
5.10.1 Script de Testes de Concorrência 48
5.10.2 Gráficos 48
5.10.3 Avaliação : Resultados e Conclusões 49

6 Implementação das Caches 53
6.1 Introdução 53
6.2 Pensamentos 53
6.2.0.1 Endpoints Simples 54
6.2.0.2 Endpoints Complexos 55
6.2.0.3 Endpoints de Resposta Direta 55
6.3 Implementação 55
6.3.1 EHCACHE 56
6.3.1.1 Informação 56
6.3.1.2 Implementação 56
6.3.2 Guava 59
6.3.2.1 Informação 59

6.3.2.2 Implementação 59
6.3.3 Caffeine 60
6.3.3.1 Informação 60
6.3.3.2 Implementação 61
6.3.4 Ignite 62
6.3.4.1 Informação 62
6.3.4.2 Implementação 63
6.3.5 Considerações entre elas e dificuldades encontradas . . 65
6.3.5.1 Caffeine e Guava 65
6.3.5.2 Ehcache 66
6.3.5.3 Ignite 67
6.3.5.4 Síntese Comparativa e Dificuldades Gerais . . 67

7 Comparação **69**
7.1 Introdução 69
7.2 Representação Gráfica 70
7.2.1 Heavy Database 71
7.2.2 Light Database 73
7.3 Recursos Utilizados 76
7.4 Análise dos Resultados 77
7.4.1 Métrica ICE 77
7.4.2 Pesos Utilizados 77
7.4.3 Heavy Database 78
7.4.3.1 Cenários de baixa concorrência 78
7.4.3.2 Cenários de média concorrência 78
7.4.3.3 Cenários de alta concorrência 79
7.4.4 Light Database 80
7.4.4.1 Cenários de baixa concorrência 80
7.4.4.2 Cenários de média concorrência 80
7.4.4.3 Cenários de alta concorrência 81
7.4.5 Analise dos Recursos 82

8 Conclusões **83**
8.1 Introdução 83
8.2 Resultados 83
8.3 Trabalho Futuro 84

Bibliografia **87**

Lista de Figuras

2.1 Fluxograma dos princípios de funcionamento da *cache*. 6
2.2 Arquitetura em duas camadas. Adaptado de [1]. 9
2.3 Arquitetura em três camadas. Adaptado de [1]. 10
2.4 Esta imagem exemplifica o fluxo de dados sem implementação da *cache*. 13

5.1 Representação esquemática da lógica e divisão do serviço. 30
5.2 Este é o esquema da base de dados utilizada. 34
5.3 Diagrama do *workflow* de uma requisição à API. 39

7.1 Gráfico de barras da base de dados Heavy cenário 1. 71
7.2 Gráfico de dispersão da base de dados Heavy cenário 1. 71
7.3 Gráfico de barras da base de dados Heavy cenário 2. 73
7.4 Gráfico de dispersão da base de dados Heavy cenário 2. 73
7.5 Gráfico de barras da base de dados Light no cenário 1. 74
7.6 Gráfico de dispersão da base de dados Light no cenário 1. 74
7.7 Gráfico de barras da base de dados Light no cenário 1. 75
7.8 Gráfico de dispersão da base de dados Light no cenário 1. 75

Lista de Excertos de Código

5.1 Exemplo de código em Python para gerar dados na versão *Heavy* da base de dados 36
5.2 Exemplo de código em Python para gerar dados na versão *Light* da base de dados 37
5.3 Codigo simplificado do endpoint. 40
5.4 Codigo para buscar a informação á Database 40

Acrónimos

SEGAL *Space & Earth Geodetic Analysis Laboratory*

GNSS *Global Navigation Satellite System*

EPOS *European Plate Observing System*

API *Application Programming Interface*

UBI Universidade da Beira Interior

CPU *Central Processing Unit*

SSD *Solid-State Drive*

UC Unidade Curricular

EI Engenharia Informática

GPU *Graphics Processing Unit*

TTL *Time To Live*

SGBD Sistema de Gestão da Base de Dados

Capítulo

1

Introdução

1.1 Enquadramento

Esta dissertação foi desenvolvida como parte do trabalho final do Mestrado em Engenharia Informática (EI) na Universidade da Beira Interior (UBI), no âmbito da Unidade Curricular (UC) "Dissertação". O principal objetivo deste trabalho é contribuir para a melhoria do projeto *European Plate Observing System* (EPOS), em particular para a comunidade de *Global Navigation Satellite System* (GNSS) parcialmente desenvolvido pelo *Space & Earth Geodetic Analysis Laboratory* (SEGAL), através da análise e implementação de um sistema de *cache* mais moderno e eficiente. Esta melhoria procura aumentar a capacidade de resposta da *Application Programming Interface* (API), desenvolvida em *Java* para o projeto, especialmente no tratamento de consultas complexas e exigentes.

1.2 Motivação

A escolha do projeto EPOS como caso de estudo para a análise de mecanismos de *cache* justifica-se, em grande medida, pelo facto de se tratar de um projeto real, amplamente utilizado e com uma comunidade ativa e de grande dimensão.

A API atualmente utilizada pela comunidade de *Global Navigation Satellite System* (GNSS), no âmbito do projeto EPOS, permite a realização de consultas por parte da comunidade de investigação, sendo geralmente acedida através de scripts em linha de comandos. Esta API possibilita ainda o desenvolvimento de plataformas web comunitárias, destinadas à monitorização, análise e disseminação dos dados recolhidos. Exemplos concretos da sua uti-

lização incluem o portal europeu disponível em `https://gnssdata-epos.oca.eu` e, no contexto nacional, o portal português acessível em `https://glass.c4g-pt.eu/`.

Esta API é responsável por lidar com um elevado volume de consultas, muitas das quais apresentam um custo computacional significativo. As consultas podem incidir sobre bases de dados locais ou remotas, com o objetivo de responder a pedidos provenientes de múltiplas fontes, tais como *web services* e aplicações. As bases de dados em questão são relativamente estacionárias, isto é, os dados nelas armazenados sofrem poucas alterações ao longo do tempo, mantendo valores maioritariamente constantes. Funcionam essencialmente como repositórios, onde a maioria das operações corresponde a inserções, sendo as atualizações e eliminações pouco frequentes.

A título de exemplo, no caso da comunidade de GNSS, o portal principal (`https://gnssdata-epos.oca.eu/`) regista cerca de dois mil inserções diárias numa tabela que já contém aproximadamente onze milhões de entradas.

Outro aspeto relevante prende-se com o histórico de desenvolvimento do projeto EPOS, que envolveu múltiplas equipas ao longo do tempo. Esta característica aproxima-o de um cenário realista, no qual as implementações existentes tendem a apresentar limitações, tanto ao nível da otimização do desempenho como da facilidade de manutenção do código.

Embora o software utilizado pela comunidade de GNSS já disponha de um mecanismo de *cache*, este encontra-se desatualizado e apresenta limitações significativas, nomeadamente em termos de desempenho em ambientes *multithreaded*, bem como dificuldades acrescidas de manutenção. Neste contexto, propõe-se a realização de uma análise teórica e empírica de diversas ferramentas e *frameworks* modernos de *cache*, com o objetivo de avaliar o seu potencial para melhorar o desempenho, a escalabilidade e a manutenibilidade do sistema.

1.3 Objetivos

O objetivo principal desta dissertação é estudar e avaliar diferentes métodos de *cache* para, posteriormente, implementar, testar e analisar essas soluções. Para alcançar este objetivo foram definidos os seguintes objetivos específicos:

1. Identificar métodos de *cache* que cumpram os requisitos do projeto;

2. Realizar uma análise teórica dos métodos, destacando as suas vantagens e desvantagens;

3. Selecionar os métodos mais relevantes e promissores para o problema em questão;
4. Criar um ambiente de execução reprodutível e confiável;
5. Desenvolver e executar testes e métricas de desempenho;
6. Analisar os resultados obtidos e apresentar uma conclusão.
7. Apresentar falhas e/ou Projetos futuros baseados nesta tese.

1.4 Organização do Documento

De forma a refletir o trabalho desenvolvido, este documento encontra-se estruturado da seguinte forma:

1. O primeiro capítulo — **Introdução** — apresenta o enquadramento do projeto, a motivação para a sua realização, os objetivos definidos e a estrutura do documento;
2. O segundo capítulo — **Conceitos Fundamentais de *Cache* e Aplicações Web** — aborda a definição de *cache*, os princípios da sua implementação, os diferentes tipos de armazenamento e as respetivas utilizações;
3. O terceiro capítulo — **Estado da Arte** — descreve o processo de pesquisa bibliográfica realizado, bem como a análise e escolha das tecnologias e ferramentas utilizadas no desenvolvimento do projeto;
4. O quarto capítulo — **Metodologia** — apresenta a metodologia adotada para a realização deste estudo;
5. O quinto capítulo — **Arquitetura do Sistema** — detalha a arquitetura do sistema desenvolvido e as principais decisões técnicas tomadas bem como o estudo da concorrência e conclusões delas;
6. O sexto capítulo — **Implementação das *Caches*** — descreve a implementação de cada *cache*, bem como as suas características e particularidades técnicas;
7. O sétimo capítulo — **Comparação** — apresenta os resultados obtidos e realiza uma análise comparativa entre eles;
8. O oitavo capítulo — **Conclusões** — sintetiza os principais contributos do estudo, resume os resultados alcançados e indica possíveis direções para trabalhos futuros.

Capítulo

2

Conceitos Fundamentais de Cache e Aplicações Web

2.1 Introdução

Este capítulo aborda o conceito de *cache* no contexto das aplicações web, explorando os seus objetivos, motivações, funcionamento e principais aplicações. Pretende-se apresentar uma visão geral do tema, destacando a importância do uso de mecanismos de *cache* na otimização de desempenho de sistemas e na redução da carga sobre repositórios de dados. Dado que a motivação desta dissertação se relaciona com o uso de *cache* em aplicações web que interagem com bases de dados, serão também introduzidos os conceitos fundamentais associados a este tipo de arquitetura.

2.2 *Cache*

2.2.1 Definição

A *cache* é um mecanismo de armazenamento rápido e temporário de dados, cujo objetivo é melhorar o desempenho de um sistema através da retenção de cópias de informação frequentemente acedida. Estas cópias são guardadas em locais de acesso mais rápido do que a fonte original, evitando a repetição de operações dispendiosas sobre fontes mais lentas, como bases de dados, discos rígidos, ou servidores remotos.

O termo *cache* tem origem no francês, derivado do verbo *cacher*, que significa "esconder" ou "guardar". O seu uso em informática remonta a 1967,

quando o editor do *IBM Systems Journal* procurava uma expressão mais adequada para descrever um *high-speed buffer*. [2]

2.2.2 Princípios de Funcionamento

O funcionamento da *cache* assenta sobre três operações fundamentais: escrita, leitura e validação.

Na fase de escrita, os dados são obtidos a partir da fonte original e armazenados no sistema de *cache* quando solicitados pela primeira vez. Durante a leitura, o sistema verifica se a informação pedida já se encontra na *cache*, e entra no processo de validação assegurando que os dados estejam atualizados. Se forem considerados válidos são devolvidos diretamente, evitando uma consulta à fonte original. Caso contrário, são substituídos por novos dados obtidos da fonte original.

A Figura2.1 ilustra de forma resumida esta arquitetura.

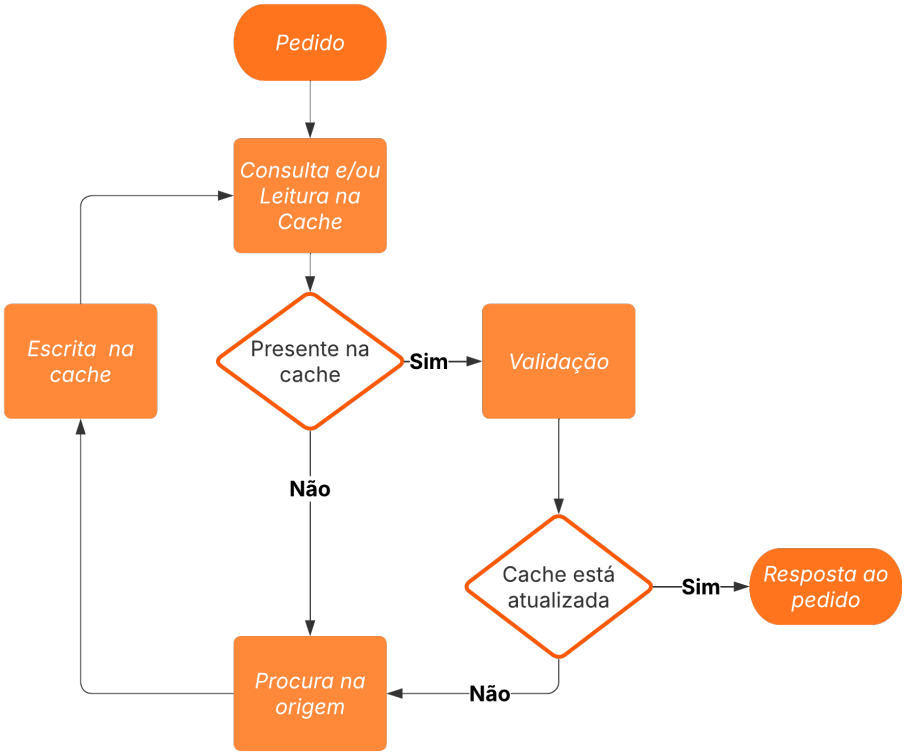

Figura 2.1: Fluxograma dos princípios de funcionamento da *cache*.

2.2.3 Benefícios e Limitações

Com base nos conceitos anteriores, é possível identificar diversas vantagens na utilização de *cache*:

1. **Redução do tempo de resposta**: o acesso aos dados é mais rápido, evitando operações repetidas e dispendiosas;

2. **Diminuição da carga sobre o sistema original**: ao armazenar localmente cópias de dados, reduz-se o número de consultas a bases de dados ou serviços remotos.

Estes benefícios permitem otimizar recursos e reduzir o custo associado ao processamento e transporte de dados. Contudo, o uso de *cache* apresenta também limitações:

1. **Desatualização de dados**: a informação armazenada pode tornar-se obsoleta, comprometendo a consistência dos resultados apresentados;

2. **Configuração complexa**: encontrar o equilíbrio ideal entre o tamanho da *cache*, a política de expiração e a frequência de atualização é um desafio;

3. **Impacto negativo no desempenho**: o uso excessivo ou uma má configuração pode aumentar o consumo de memória e processamento, degradando o desempenho global do sistema.

2.3 Tipos Clássicos de Armazenamento em *Cache*

A *cache* pode utilizar diferentes meios de armazenamento, consoante o tipo de dados e o nível de desempenho pretendido:

1. **Memória**: armazena dados em RAM, proporcionando acesso extremamente rápido. É a abordagem mais comum em sistemas modernos;

2. **Disco ou *Solid-State Drive* (SSD)**: utiliza espaço em disco para armazenar dados. Embora mais lenta do que a memória, é adequada para informação acedida com menor frequência;

3. **CPU**: recorre a pequenas áreas de memória ultra-rápida integradas no processador. É a forma mais eficiente, mas também a mais limitada em capacidade.

2.3.1 Aplicações e Usos da Cache

Os diferentes métodos de armazenamento em *cache* são aplicados conforme as necessidades de desempenho e o contexto da aplicação. Navegadores web, por exemplo, utilizam *cache* em disco ou em SSD para armazenar páginas e recursos estáticos, como imagens e ficheiros CSS, acelerando carregamentos subsequentes. [3]

Em aplicações que exigem respostas rápidas, como APIs e Web Services, a *cache* em memória é amplamente utilizada para guardar respostas a pedidos recorrentes, reduzindo a latência e a carga sobre servidores externos. De forma semelhante, sistemas de inteligência artificial recorrem à *cache* para armazenar resultados de cálculos frequentemente reutilizados, tanto em memória principal como diretamente na *Graphics Processing Unit* (GPU), otimizando processos de treino.

Ao nível do hardware, a *cache* do *Central Processing Unit* (CPU) (L1, L2 e L3) desempenha um papel fundamental na aceleração das operações de processamento, ao minimizar o acesso à memória principal e melhorar a eficiência computacional.

2.4 Arquiteturas de Desenvolvimento *Web*

A arquitetura em camadas (*tiers*) organiza os sistemas em subsistemas hierarquizados segundo a sua função. As configurações mais comuns são as arquiteturas em duas e três camadas, sendo as de múltiplas camadas extensões menos frequentes da última. [4]

As camadas são normalmente classificadas como:

- **Camada de interface**: responsável pela interação com o utilizador, exibindo e recolhendo dados;
- **Camada de aplicação** (ou lógica): processa as informações recebidas, podendo realizar operações sobre os dados;
- **Camada de dados**: armazena e gere a informação, podendo corresponder a uma base de dados relacional (como PostgreSQL ou MySQL), um sistema *NoSQL* (como MongoDB) ou um servidor de ficheiros.

2.4.1 Arquitetura em Duas Camadas

A arquitetura em duas camadas segue o modelo cliente-servidor, constituída por uma camada de interface e uma de dados, o cliente comunica diretamente com a base de dados, podendo realizar algum processamento básico.

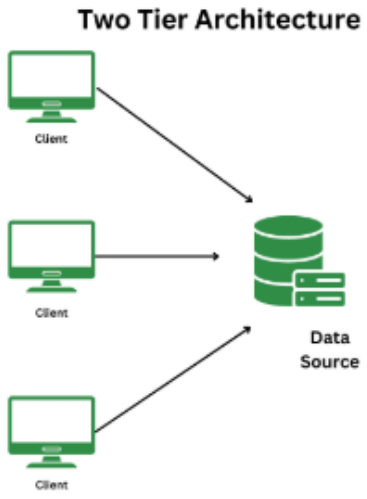

Figura 2.2: Arquitetura em duas camadas. Adaptado de [1].

2.4.2 Arquitetura em Três Camadas

Na arquitetura em três camadas, toda a comunicação entre o cliente e os dados passa pela camada de aplicação. Este modelo separa claramente as responsabilidades, facilitando a escalabilidade, a manutenção e a segurança esta está ilustrado na 2.4.2.

2.5 Modelos de *Cache* em Aplicações Web

O uso de *cache* é essencial em arquiteturas modernas, permitindo reduzir latências, otimizar desempenho e diminuir a carga sobre servidores e bases de dados. A escolha do modelo adequado depende da arquitetura e dos objetivos da aplicação. [5]

2.5.1 *Cache* em Três Camadas

A *cache* pode ser aplicada em diferentes camadas de uma arquitetura de três níveis:

- **Frontend (camada de apresentação)**: armazena respostas prontas, como páginas HTML, JSON ou recursos estáticos;
- **Middleware (camada de aplicação)**: guarda resultados de cálculos ou consultas complexas;

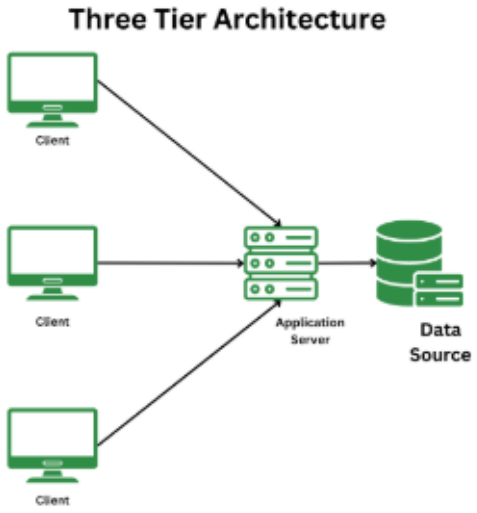

Figura 2.3: Arquitetura em três camadas. Adaptado de [1].

- **Backend (camada de dados)**: otimiza consultas frequentes à base de dados, através de índices ou *queries* pré-processadas.

Esta abordagem proporciona maior escalabilidade e flexibilidade, mas aumenta a complexidade da gestão e os riscos de inconsistência entre as diferentes *cache*s.

2.5.2 *Cache* em Duas Camadas

Em arquiteturas de duas camadas, a *cache* pode ser aplicada:

- No **cliente**, através de mecanismos como *cookies* ou *localStorage*;
- No **servidor**, armazenando resultados de consultas repetidas.

Esta abordagem é mais simples de implementar e manter, mas apresenta limitações em termos de escalabilidade e resiliência uma vez que o servidor pode tornar-se um ponto único de falha. [6]

2.5.3 Boas Práticas de Implementação

Para garantir uma utilização eficaz e segura da *cache*, devem ser seguidas várias boas práticas:

- Definir tempos de expiração adequados (*Time To Live* (TTL)) de acordo com a frequência de atualização dos dados;
- Selecionar cuidadosamente os dados a armazenar, evitando informações sensíveis ou voláteis;
- Implementar políticas de substituição (*eviction policies*) para evitar saturação;
- Garantir segurança através de encriptação e controlo de acessos;
- Monitorizar o desempenho e automatizar a limpeza de dados desatualizados;
- Diferenciar entre conteúdo estático (prioritário para *cache*) e dinâmico (que requer estratégias específicas).

Estas práticas asseguram maior eficiência e uma melhor experiência do utilizador. [7]

2.5.4 Coerência dos Dados

A coerência dos dados em sistemas com *cache* é essencial para garantir a integridade das informações. Entre as estratégias mais comuns destacam-se:

- ***ETag***: cada versão de um recurso recebe um identificador único; o servidor usa-o para indicar se os dados foram modificados [8].
- ***Last-Modified***: o servidor regista a data da última atualização e apenas envia novamente os dados se ocorreram alterações [9].
- ***Write-Through***: todas as operações de escrita atualizam simultaneamente a *cache* e o armazenamento principal, assegurando consistência imediata [10].

A escolha da estratégia depende do tipo de aplicação, da frequência de atualização e da criticidade dos dados [11].

2.5.5 Caso de Estudo:EPOS

O projeto EPOS é uma iniciativa europeia que visa estabelecer uma infraestrutura de pesquisa multidisciplinar e distribuída, cujo propósito é facilitar o uso integrado de dados, ferramentas de análise e instalações da comunidade científica das geociências na Europa. Este projeto tem como objetivo a distribuição e armazenamento de dados GNSS.

Esta base de dados contém um número elevado de tabelas com relações entre si, aumentando a necessidade do uso de todos os tipos de *joins*. Para além desse ponto, as próprias tabelas contêm vários campos *text* e *varchar*, sendo ainda importante realçar a possibilidade de filtros específicos e opcionais como, por exemplo, do tipo *"Date"* e *"Like"*, entre outros nas *querrys*. Isto gera um aumento ainda mais severo no custo computacional e no tempo de resposta no servidor onde está alocado o Sistema de Gestão da Base de Dados (SGBD). De notar que quase todos os exemplos supracitados possuem um custo computacional mais elevado que o habitual.

O fluxo de dados começa com um cliente, enviando uma requisição *HTTPS* do tipo *GET* para obter dados ou *POST* para enviar informações. Essa comunicação é feita através do protocolo seguro *HTTPS*, que garante a criptografia e integridade dos dados durante a transmissão. A requisição chega até uma API construída com *Java* e *Jakarta*, onde é encaminhada para o *endpoint* correspondente.

Quando a aplicação precisa de aceder ou de manipular dados, a API comunica com a base de dados utilizando instruções *SQL*, por intermédio de um SGBD, o *PostgreSQL*. Os dados que regressam pela base de dados são mapeados para objetos *Java*, um processo facilitado por ferramentas como a JPA (*Java Persistence API*), que convertem os registos da base de dados em entidades compreensíveis para a aplicação.

A próxima etapa envolve o *parsing* e a serialização desses objetos para formatos legíveis pelo cliente, tais como *JSON*, *XML*, *CSV* ou outros formatos específicos. Este processo permite que os dados sejam estruturados de maneira eficiente para serem consumidos pelo cliente. Por fim, a resposta é enviada de volta ao cliente através de uma resposta *HTTPS*, concluindo o ciclo de comunicação. Todo o fluxo é projetado para garantir a segurança, eficiência e interoperabilidade entre os diferentes sistemas envolvidos. Este fluxo pode ser visualizado de forma esquemática na imagem 2.5.5.

A escolha deste projeto foi motivada pela sua relevância prática e complexidade arquitetural, características que tornaram a análise do desempenho de *caches* um tema ainda mais pertinente. O fato de se tratar de um sistema legado, com soluções datadas, reforça a importância de estudos que promovam a modernização e a otimização dos processos de resposta de dados.

Como resultado, espera-se que este estudo contribua para melhorar o desempenho geral do sistema EPOS, oferecendo *insights* sobre boas práticas de otimização e fornecendo uma base sólida para futuras melhorias na infraestrutura. Além disso, a possibilidade de aplicar as métricas e ferramentas desenvolvidas a outros projetos semelhantes abre novas perspetivas para a evolução e inovação em projetos de geociências distribuídos.

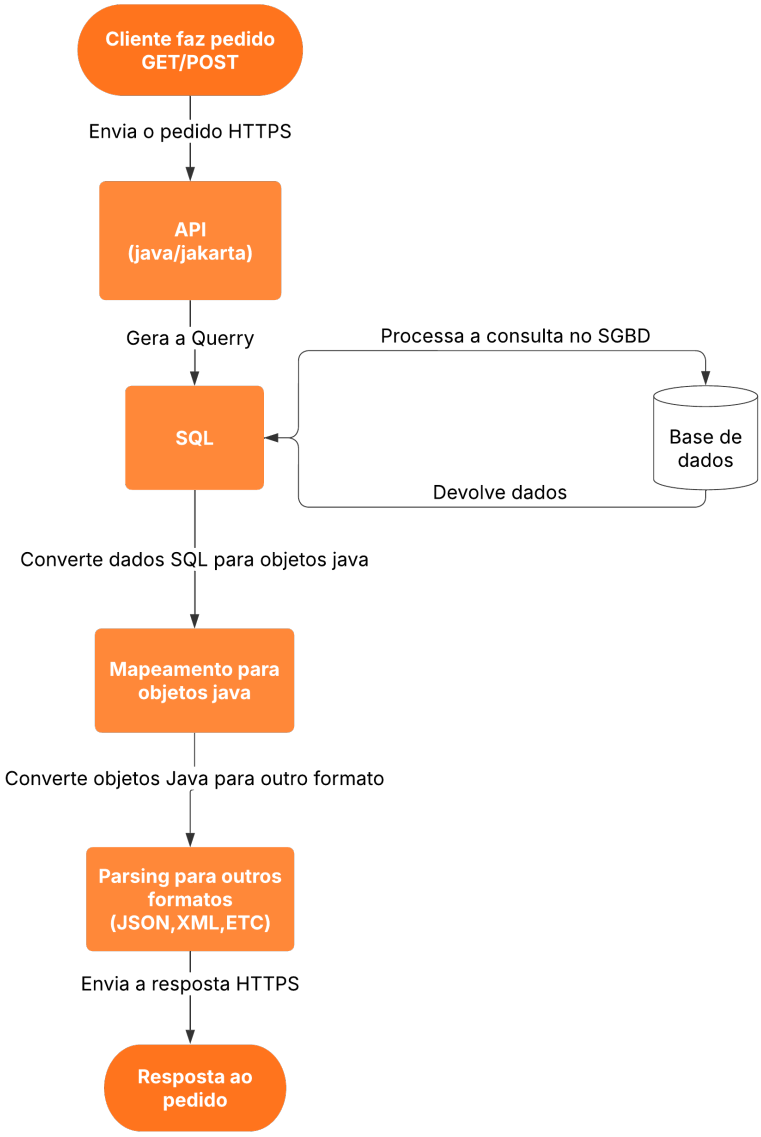

Figura 2.4: Esta imagem exemplifica o fluxo de dados sem implementação da *cache*.

2.6 Conclusões

Neste capítulo foi apresentada uma visão abrangente sobre o conceito de *cache*, os seus princípios de funcionamento, vantagens, limitações, tipos de armazenamento e aplicações práticas. Foi também explorada a sua integração em diferentes arquiteturas de sistemas *web*, destacando a importância

de uma implementação criteriosa para garantir equilíbrio entre desempenho, consistência e escalabilidade. Por fim, foi apresentado o projeto EPOS e respetivo enquadramento. Conclui-se que a *cache* não deve ser vista apenas como uma tecnologia isolada, mas como um conceito essencial de otimização que se adapta a múltiplos contextos e níveis da arquitetura de sistemas modernos.

Capítulo

3

Estado da Arte

3.1 Introdução

Neste capítulo, apresenta-se o método usado para estudar o estado da arte relativamente ao estudo de caches e métricas bem como aos métodos de *cache* aplicados a aplicações *web*, com ênfase especial nas *frameworks* desenvolvidas para Java. O objetivo é fornecer uma visão ampliada das soluções existentes, das metodologias de avaliação de desempenho e das estratégias de implementação de *cache* em sistemas contemporâneos. Tal análise permite identificar lacunas na literatura e fundamentar a necessidade de investigação mais direcionada acerca do impacto das diferentes *frameworks* de *cache* na performance de aplicações *web*.

3.2 Método de Pesquisa

A pesquisa foi conduzida com o objetivo de identificar e analisar estudos recentes relacionados à implementação e avaliação de técnicas de *cache* em aplicações web desenvolvidas em Java. O foco principal foi compreender as soluções, metodologias e métricas de desempenho utilizadas em contextos empresariais e de larga escala.

3.2.1 Strings de Pesquisa

Para a busca dos artigos, foram definidas *strings* que combinassem termos relacionados a *cache*, desempenho e aplicações web em Java. As principais combinações utilizadas foram:

- "Comparative Study of Distributed Caching Solutions in Enterprise Java Applications"
- "Performance Analysis of In-Memory Caching Systems for Web Applications"
- "Java cache technologies for web applications"
- "Java cache methods for web applications"
- "Java cache best practices"
- "Java cache design for web applications"
- "Performance evaluation of web applications"
- "Benchmarks in cache software"

Essas combinações foram escolhidas para integrar estudos comparativos, análises de desempenho e boas práticas de implementação de mecanismos de *cache* voltados para aplicações web desenvolvidas em Java.

3.2.2 Repositórios de Pesquisa

A pesquisa foi realizada em diversos repositórios científicos e bases de dados digitais reconhecidas, incluindo:

- **b-on (Biblioteca do Conhecimento Online)**
- **arXiv**
- **ScienceDirect**
- **Elsevier**
- **Google Scholar**
- **ResearchGate**

Foram considerados artigos publicados a partir de 2020, de forma a garantir a atualidade das soluções e tecnologias analisadas, uma vez que o ecossistema de desenvolvimento Java e as ferramentas de *cache* evoluem rapidamente e constantemente.

3.2.3 Critérios de Seleção

Durante a triagem inicial, foi identificado um número considerável de resultados. No entanto, muitos dos artigos encontrados apresentavam caráter excessivamente teórico, fugindo do escopo definido ou abordavam conceitos de *cache* não aplicáveis a aplicações web. Assim, foram excluídos estudos que:

- Não envolviam diretamente a linguagem Java;
- Tratavam de *cache* em contextos não relacionados à web (por exemplo, avaliações de LLMs e detecção de erros);
- Não apresentavam métricas de desempenho mensuráveis ou métodos de avaliação experimental.

3.2.4 Critérios de Inclusão

A revisão sistemática concentrou-se em artigos que:

- Abordassem implementações práticas de *cache* em Java;
- Descrevessem métodos de avaliação de desempenho;
- Utilizassem métricas de eficiência, tais como *hit rate*, latência, tempo de resposta e consumo de recursos.

3.2.5 Síntese dos Resultados

Após a aplicação dos critérios de inclusão e exclusão, obteve-se um conjunto reduzido, porém relevante, de artigos que serviram de base para a análise comparativa. Esses trabalhos forneceram informações úteis sobre o impacto do uso de *cache* na eficiência de aplicações web em Java, permitindo identificar padrões de boas práticas e desafios ainda existentes no campo.

3.3 Revisão dos estudos mais importantes

3.3.1 A Performance Comparison of Java Cache Memory Implementations

O estudo de Kabakuş (2020) [12], intitulado *A Performance Comparison of Java Cache Memory Implementations*, constitui referência central para a compreensão do comportamento de diferentes soluções de *cache* em Java. O autor

realiza uma comparação entre diversas implementações, analisando métricas como tempo de execução e consumo de memória. Conclui-se que o desempenho de uma solução de *cache* depende fortemente do seu desenho arquitetónico e do tipo de operação executada, não existindo implementação universalmente superior. Todavia, o estudo apresenta limitações importantes: as bibliotecas avaliadas encontram-se desatualizadas. Os testes foram conduzidos em contexto simplificado, sem contemplar cenários complexos de aplicações *web* com múltiplos acessos concorrentes. Apesar dessas limitações, o trabalho fornece uma base metodológica sólida e constitui o ponto de partida relevante para a análise comparativa de *frameworks* de *cache* mais modernas.

3.3.2 An Overview of Analysis Methods and Evaluation Results for Caching Strategies

Hasslinger et al. (2023) [13], no estudo *An Overview of Analysis Methods and Evaluation Results for Caching Strategies*, apresenta uma revisão abrangente dos métodos de análise e resultados de avaliação aplicados a diferentes estratégias de *caching*. Os autores discutem políticas de evacuação de itens, como FIFO, LRU e LFU, e propõem modelos analíticos para a avaliação do desempenho de sistemas de *cache* em distintos contextos. Destaca-se, nesse estudo, a necessidade de considerar múltiplas métricas não apenas a taxa de acerto (*hit rate*), mas também fatores como custo de atualização, armazenamento, latência e tráfego gerado. Apesar do seu caráter predominantemente teórico e da ausência de aplicação prática em ambientes Java, o estudo fornece uma estrutura metodológica útil para a compreensão e comparação das políticas de *cache*, apoiando a definição das métricas utilizadas nesta dissertação.

3.3.3 JVM Optimization: An Empirical Analysis of JVM Configurations for Enhanced Web Application Performance

Noetzold et al. (2024), [14] em *JVM Optimization: An Empirical Analysis of JVM Configurations for Enhanced Web Application Performance*, aborda a otimização da Máquina Virtual Java (JVM) e analisa como diferentes configurações influenciam o desempenho de aplicações *web*. Embora o foco principal não seja diretamente o *cache*, o estudo evidencia que fatores como tipo de *Garbage Collector*, alocação de memória e configuração de *threads* impactam significativamente o comportamento de sistemas que dependem fortemente de *cache*. Os resultados indicam melhorias significativas no desempenho, in-

cluindo redução do tempo de resposta e da utilização de CPU. Este trabalho reforça a necessidade de considerar o ambiente de execução na avaliação de *frameworks* de *cache*, demonstrando que a eficiência do *cache* não depende apenas de seu algoritmo interno, mas também da sua interação com o sistema hospedeiro.

3.3.4 Redis vs. Memcached in Microservices Architectures: Caching Strategies

O artigo de Kanthed (2023), [15] *Redis vs. Memcached in Microservices Architectures: Caching Strategies*, apresenta análise comparativa entre Redis e Memcached em arquiteturas de microserviços, considerando parâmetros como latência, *throughput* e consumo de memória, além da integração com ambientes de containerização, como Kubernetes. Apesar do foco em *caches* externos (*in-memory*) e não em *frameworks* Java embutidas, o estudo evidencia a importância de compreender o impacto das estratégias de armazenamento, replicação e persistência em cenários distribuídos. Tal constatação revela uma lacuna na literatura: a escassez de estudos que comparem *frameworks* internas de *cache* para Java em aplicações web tradicionais sob cargas de trabalho realistas e ambientes concorrentes.

3.3.5 Database Loading Strategies for an In-Memory Cache in Java

Henckel e Söderberg (2022), [16] na obra *Database Loading Strategies for an In-Memory Cache in Java*, investigam estratégias de carregamento de dados em memória (*in-memory caching*) aplicadas a sistemas Java, analisando métodos de sincronização entre o *cache* e a base de dados subjacente para reduzir latência e melhorar escalabilidade. O estudo avalia técnicas como *lazy loading*, *write-through* e *write-behind*, examinando o seu impacto sobre latência, consistência e consumo de recursos.

Os resultados indicam que o *lazy loading* proporciona ganhos expressivos em aplicações com padrões de acesso previsíveis, enquanto *write-through* e *write-behind* são mais adequados para sistemas que exigem elevada consistência e atualização contínua de dados. Observou-se ainda que o aumento do tamanho do *cache* nem sempre resulta em melhoria linear do desempenho, especialmente quando o custo de sincronização com a base de dados é elevado. Limitações do estudo incluem a restrição a cenários limitados, ausência de avaliação de diferentes *frameworks* Java modernas e foco exclusivo em bancos de dados relacionais, sem explorar ambientes distribuídos ou mi-

croserviços. Apesar disso, a pesquisa evidencia a importância da interação entre *cache* e sistema de persistência, fator crítico na performance de aplicações *web* em Java.

3.3.6 Caching Strategy for Web Application – A Systematic Literature Review

Zulfa, Hartanto e Permanasari (2020), no estudo *Caching Strategy for Web Application – A Systematic Literature Review* [17] realizam uma revisão sistemática da literatura com o objetivo de identificar, classificar e comparar as principais estratégias de *caching* aplicadas a aplicações *web*. Os autores analisam três abordagens predominantes: *web caching*, *web prefetching* e *application-level caching*, discutindo seus mecanismos, métricas de avaliação e contextos de uso.

A pesquisa revela que o *web caching* se concentra na armazenagem de objetos frequentemente solicitados para reduzir latência e tráfego de rede, enquanto o *prefetching* visa prever e carregar antecipadamente recursos futuros, mitigando o tempo de resposta. Já o *application-level caching* opera em nível de aplicação, otimizando o acesso a dados e operações custosas. O estudo destaca que, embora todas as técnicas compartilhem o objetivo de melhorar o desempenho, diferem significativamente nas suas políticas de substituição, previsibilidade de acesso e impacto sobre a consistência dos dados.

Apesar das suas contribuições, a obra apresenta algumas limitações, especialmente no recorte temporal adotado restrito aos últimos dez anos que, embora represente um período considerável, pode incluir abordagens já desatualizadas frente às evoluções recentes em tecnologias de *cache*. Além disso, o estudo não explora de forma aprofundada tendências emergentes, como *edge caching* e o uso de aprendizado de máquina em políticas de substituição. Ainda assim, a obra de Zulfa et al. (2020) constitui referência importante por consolidar o estado da arte sobre estratégias de *caching* em aplicações *web*, fornecendo a base teórica sólida para futuras investigações experimentais e comparativas em ambientes Java e arquiteturas de microserviços, evidenciando mais uma vez a lacuna existente na literatura quanto à avaliação prática e integrada dessas abordagens.

3.3.7 Conclusão

A análise da literatura evidencia lacunas significativas nas comparações diretas entre mecanismos de *cache* em Java. Observa-se que muitos estudos são excessivamente teóricos ou apresentam foco restrito em aspetos específicos, como configurações de TTLs, sem abordar comparações abrangentes entre

diferentes implementações. É certo que diversas pesquisas estão desatualizadas ou limitadas a contextos reduzidos, o que compromete a generalização dos resultados.

Ademais verifica-se que o conceito de *cache* é amplo e, por vezes, gera ruído nas buscas e seleções bibliográficas, dificultando a consolidação de um panorama comparativo consistente. Nesse sentido, esta dissertação propõe-se servir como um guia metodológico para o teste e a comparação de soluções de *cache* em aplicações web Java, contribuindo para suprir parte dessa lacuna identificada na literatura.

3.4 Bibliotecas de *cache* para *Java*

Nesta secção apresentam-se as principais bibliotecas de *cache* disponíveis para Java, com destaque para as suas características arquitetónicas e funcionalidades específicas. O objetivo é fornecer uma visão geral das soluções utilizadas na prática, preparando o terreno para a análise detalhada de cada uma delas.

3.4.1 *Guava*

A biblioteca *Guava*, desenvolvida pelo Google, fornece soluções de *cache* eficientes para Java, permitindo armazenamento temporário de dados em memória para otimização do desempenho das aplicações. As *caches* da *Guava* são locais, existindo apenas durante a execução da aplicação, sem persistência em disco ou servidores externos.

A biblioteca possibilita a configuração de parâmetros como tamanho máximo da *cache*, TTL dos itens e comportamentos específicos de remoção. Internamente, utiliza uma estrutura similar a *ConcurrentHashMap*, garantindo acesso e modificações *thread-safe*. Para suportar referências fracas (*weak references*) ou suaves (*soft references*), emprega filas de referência que auxiliam o *Garbage Collector* na liberação eficiente de memória.

A *Guava* implementa políticas de remoção por tamanho, tempo ou peso total da *cache*, além de permitir atualização automática de entradas (*refreshAfterWrite*) e registro de *RemovalListener* para monitoramento de mudanças. Essas funcionalidades permitem otimização do uso de memória e manutenção da consistência dos dados [18].

3.4.2 *Apache Ignite*

O *Apache Ignite* é uma plataforma de computação distribuída centrada em memória, combinando armazenamento em memória e disco com processamento distribuído. Os dados são mantidos em regiões de memória pré-alocadas fora do *heap* da JVM, reduzindo pausas no *Garbage Collector*. A persistência nativa utiliza *Write-Ahead Logging (WAL)*, garantindo durabilidade e consistência.

O Ignite suporta processamento distribuído, consultas paralelas e oferece APIs para computação, serviços, aprendizado de máquina e *streaming* em tempo real. A consistência em ambientes distribuídos é assegurada pelo protocolo *Two-Phase Commit* [19].

3.4.3 *Ehcache*

O *Ehcache* é uma biblioteca de *cache open-source* para Java, amplamente utilizada para melhorar desempenho de aplicações. A sua arquitetura modular permite armazenamento em memória (*MemoryStore*), fora do *heap* (*OffHeapStore*) ou em disco (*DiskStore*), possibilitando hierarquias de *cache* que equilibram desempenho e capacidade. A configuração é realizada programaticamente ou via arquivos XML [20].

3.4.4 *Cache4j*

O *Cache4j* oferece API simples para armazenamento em memória de objetos Java, com suporte a ambientes multithread e algoritmos de remoção LFU, LRU e FIFO. Contudo, encontra-se desatualizado há cerca de 18 anos, sem suporte ativo, tornando-o desaconselhável para aplicações modernas [21].

3.4.5 *Caffeine*

O *Caffeine* é uma biblioteca de alto desempenho inspirada na API da *Guava*, oferecendo algoritmos avançados de gestão de entradas, como *Window TinyLFU*. Permite configuração de tamanho máximo, TTL e estratégias de remoção, além de monitoramento detalhado de métricas de desempenho, proporcionando alta taxa de acertos e baixa latência [22].

3.4.6 *Payara*

O *Payara Cache* baseado em *URL* armazena respostas de requisições para reduzir carga do servidor e acelerar tempo de resposta. Contudo, não é indicado

para operações dinâmicas, pois a variabilidade dos dados comprometeria a precisão das respostas armazenadas [23].

3.4.7 *Redis*

O *Redis* é um sistema de armazenamento de dados em memória bastante popular, utilizado como *cache* distribuída, *message broker* e base de dados *NoSQL*.

Apesar do seu elevado desempenho e ampla adoção, o Redis não disponibiliza uma integração nativa com a JVM, dependendo de clientes externos para comunicação com aplicações Java. Esta característica introduz sobrecarga de serialização e comunicação em rede, o que pode ser um fator limitativo em cenários onde uma solução de cache estritamente integrada ao ecossistema Java é preferível [24].

3.4.8 Escolhas para o estudo

Das *frameworks* de cache anteriormente apresentadas, foram selecionadas para este estudo as seguintes: *Ehcache, Apache Ignite, Guava e Caffeine*.

A *framework Cache4j* foi excluída da análise principalmente por não receber atualizações recentes, o que levanta preocupações quanto à sua manutenção, segurança e compatibilidade com tecnologias atuais.

A solução de cache disponibilizada pelo *Payara* também foi descartada, uma vez que o seu funcionamento é baseado em URLs, não se adequado ao suporte de pesquisas dinâmicas, características fundamentais da *API* em estudo.

Por fim, o *Redis* não foi incluído na seleção pois não oferece suporte nativo à linguagem *Java*, tratando-se de uma solução de cache distribuída de uso mais genérico e multiplataforma. Em substituição, optou-se pelo *Apache Ignite*, que apresenta propósito e lógica semelhantes e é desenvolvido com foco no ecossistema Java, oferecendo melhor integração com aplicações construídas nesta tecnologia.

A tabela 3.1 apresenta uma síntese comparativa dos escolhidos para o estudo e mais algumas caraterísticas das mesmas.

Tabela 3.1: Características de Software de Cache

Funcionalidades/Cache	**Ehcache**	**Caffeine**	**Guava**	**Ignite**
Armazenamento	Heap, off-Heap, Disco	Heap	Heap	RAM Distribuída
Persistência	Sim	Não	Não	Sim
Cache MISS	Não	Sim	Sim	Sim
Tamanho da Cache	MB	Entradas/MB	Entradas/MB	Máx. 64 kb por entrada
TTL	Sim	Sim	Sim	Não
Algoritmos de evicção	LRU, LFU, FIFO	W-TinyLRU	LRU	Nenhum
Implementação	Média	Simples	Simples	Complexa
Documentação	Boa	Razoável	Razoável	Fraca

3.4.9 Conclusão

A revisão do estado da arte evidenciou que o desempenho de sistemas de *cache* em aplicações Java depende de múltiplos fatores, incluindo arquitetura interna, políticas de substituição, estratégias de integração com bases de dados e configuração do ambiente de execução.. Além disso, foi possível identificar lacunas relevantes na literatura, especialmente no que se refere à análise comparativa de *frameworks* modernas de *cache* em cenários de alta concorrência e aplicações web reais.

A apresentação das principais bibliotecas de *cache* disponíveis para Java complementa essa análise fornecendo um panorama das soluções utilizadas na prática. Cada biblioteca possui características específicas e funcionalidades distintas refletindo diferentes necessidades de desempenho, escalabilidade e integração com sistemas existentes. Esse mapeamento das ferramentas disponíveis estabelece uma base sólida para a investigação experimental proposta, permitindo avaliar de forma crítica o impacto de diferentes *frameworks* na performance de aplicações *web*.

Em síntese, a combinação da revisão teórica com a análise das bibliotecas disponíveis fornece subsídios essenciais para compreender os desafios e as oportunidades na implementação de estratégias de *cache* eficientes em aplicações Java modernas.

Capítulo

4

Metodologia

4.1 Introdução

Neste capítulo descreve-se a metodologia adotada nesta dissertação. Será apresentado de forma detalhada os procedimentos seguidos, incluindo o planeamento e criação dos ambientes de execução, a implementação dos mecanismos de *cache*, a realização dos testes de desempenho e a definição das métricas utilizadas para a análise dos resultados.

4.2 Ambiente de Execução

O planeamento do ambiente de execução teve como objetivo garantir a reprodutibilidade e consistência dos testes, assegurando que os resultados possam ser repetidos em condições equivalentes e permite futuras investigações com base nos mesmos pressupostos.

A *database* foi criada e controlada num *container* Docker, enquanto a API foi executada nativamente no *host*, com recursos limitados, de modo a manter o controlo sobre o consumo de recursos e a estabilidade do ambiente experimental. Alternativamente, a execução da API também poderia ser realizada num *container* Docker, sem impacto significativo no controlo experimental.

4.2.1 Grupo de Controlo

Para análise dos resultados, foi criado um grupo de controlo (*api-base*), correspondente à versão da API sem qualquer mecanismo de *cache* implementado, representando a versão base do sistema. Nesta versão inicial realizaram-

se os testes preliminares, que permitiram definir o nível de concorrência e determinar as métricas de avaliação mais relevantes.

4.3 Definição das Métricas

Durante os testes iniciais, estabeleceram-se critérios claros e consistentes para a avaliação do desempenho do sistema. Com esse propósito, foram definidas as seguintes métricas principais: utilização da CPU, uso de memória RAM, desempenho do disco, qualidade das respostas e quantidade de respostas.

A utilização da CPU avalia o percentual de uso do processador, identificando possíveis gargalos relacionados à capacidade de processamento e medindo a eficiência na utilização dos recursos disponíveis. O uso de memória RAM monitoriza o consumo de memória durante a execução, permitindo detetar sobrecarga ou desperdício de recursos. O desempenho do disco avalia as operações de leitura e escrita, incluindo as taxas de IOPS (*Input/Output Operations per Second*), fornecendo informações sobre a eficiência de acesso e armazenamento de dados. A qualidade das respostas verifica a precisão das respostas, garantindo que os dados armazenados na *cache* mantenham-se válidos. Por fim, a quantidade de respostas avalia a relação entre pedidos e respostas válidas, analisando a robustez e estabilidade da API.

A análise conjunta dessas métricas, em correlação com o tempo médio de resposta do sistema, permite compreender o impacto direto das diferentes soluções de *cache* no desempenho global [25].

4.3.1 Concorrência

Para avaliar o desempenho da API, foram realizados diversos testes de performance com o objetivo de identificar níveis de concorrência adequados e evitar execuções redundantes em cenários sem variação significativa de resultados.

O termo concorrência refere-se ao número de chamadas simultâneas feitas à API. À medida que esse número aumenta, cresce a demanda sobre os recursos do sistema podendo ocorrer degradação de desempenho.

Para determinar os pontos ideais de carga foram realizadas dez mil chamadas à API, distribuídas por diferentes níveis de concorrência. Em cada configuração, executaram-se mil chamadas idênticas, mantendo um número fixo de execuções paralelas por exemplo: dez chamadas simultâneas. Sempre que uma chamada era concluída, outra era iniciada, garantindo que o nível de concorrência permanecesse constante durante todo o teste.

A cada nova execução, o nível de concorrência era incrementado em dez, variando de dez em dez até cem chamadas simultâneas. Este procedimento permitiu simular diferentes níveis de carga no servidor, representando cenários reais com maior ou menor frequência de requisições.

4.3.2 Implementação dos Mecanismos de *Cache*

Após a seleção dos mecanismos de *cache*, cada um foi implementado na API, originando diferentes versões do sistema. O objetivo desta etapa foi avaliar experimentalmente o impacto de cada mecanismo no desempenho global da aplicação, tanto em termos de utilização de recursos como de tempo de resposta.

Quando a implementação de determinado mecanismo não se revelou viável, a situação foi registada para referência em trabalhos futuros e eventuais otimizações.

4.4 Criação dos Testes

Concluída a implementação das versões da API, iniciou-se a fase de testes, que consistiu na simulação de múltiplos pedidos equivalentes entre as diferentes versões. Os testes foram conduzidos mantendo constantes a quantidade, a complexidade e a frequência das requisições, bem como o período de execução e as condições experimentais, garantindo consistência e comparabilidade dos resultados.

Durante os testes, recolheram-se diversos dados, posteriormente utilizados para a elaboração de gráficos que auxiliaram a análise e sustentaram o processo de tomada de decisão.

4.5 Análise dos Dados

Após a recolha das métricas, procedeu-se à análise detalhada dos dados obtidos, com o objetivo de avaliar o impacto das diferentes implementações de *cache* no desempenho da API, com base nos parâmetros definidos.

Compararam-se as versões da API que incorporavam distintos mecanismos de *cache*, considerando todos os indicadores de desempenho. A análise permitiu identificar gargalos, áreas de melhoria e comportamentos inesperados, contribuindo para compreender o comportamento do sistema sob diferentes níveis de carga.

Com base nos resultados, elaborou-se este relatório, que sintetiza as observações sobre o desempenho de cada versão da API, destacando as imple-

mentações mais eficientes e as limitações identificadas, servindo de base para futuras recomendações.

4.5.1 Conclusão

A metodologia adotada permitiu estruturar o processo experimental de forma rigorosa e reprodutível, assegurando a validade dos resultados obtidos. O planeamento cuidadoso do ambiente de execução, a definição criteriosa das métricas e o controlo das variáveis experimentais garantiram a fiabilidade das medições e a comparabilidade entre as diferentes versões da API.

A execução sistemática dos testes e a análise detalhada dos dados possibilitaram uma compreensão clara do impacto de cada mecanismo de *cache* no desempenho do sistema. Esta abordagem metodológica forneceu uma base sólida para a avaliação objetiva das soluções implementadas e servirá de referência para trabalhos futuros que pretendam aprofundar a otimização de desempenho em arquiteturas semelhantes.

Capítulo

5

Arquitetura do Sistema

5.1 Lógica do Serviço

A arquitetura do sistema desenvolvido para a análise de desempenho foi concebida com base no modelo em três camadas, conforme ilustrado na secção 2.4. Esta abordagem promove a separação de responsabilidades, facilita a manutenção, aumenta a escalabilidade do sistema e simplifica a deteção de potenciais gargalos.

A primeira camada, designada por Backend ou Camada de Dados, é constituída por um container Docker configurado com uma instância do SGBD PostgreSQL 15. Nesta instância estão configuradas duas bases de dados.

A segunda camada corresponde à API, implementada com a framework Spring Boot e executada localmente no ambiente de desenvolvimento IntelliJ.

Esta camada é responsável pela mediação das comunicações entre a aplicação e a base de dados, através da definição e gestão de *endpoints*, bem como pela construção e execução das queries SQL necessárias e a formatação da resposta no formato desejado. Além disso, a API gera automaticamente um ficheiro no formato CSV contendo métricas essenciais para a análise detalhada do desempenho do sistema.

A terceira camada tem como função simular os clientes e os pedidos ao serviço. Esta componente foi desenvolvida através de *scripts* em Python, executados no PowerShell, e tem como objetivo principal a geração pseudoaleatória de requisições à API. Desta forma, é possível avaliar a robustez e a capacidade de resposta do sistema sob diferentes níveis de carga.

A figura 5.1 apresenta uma visão esquemática da lógica e da estrutura do serviço implementado.

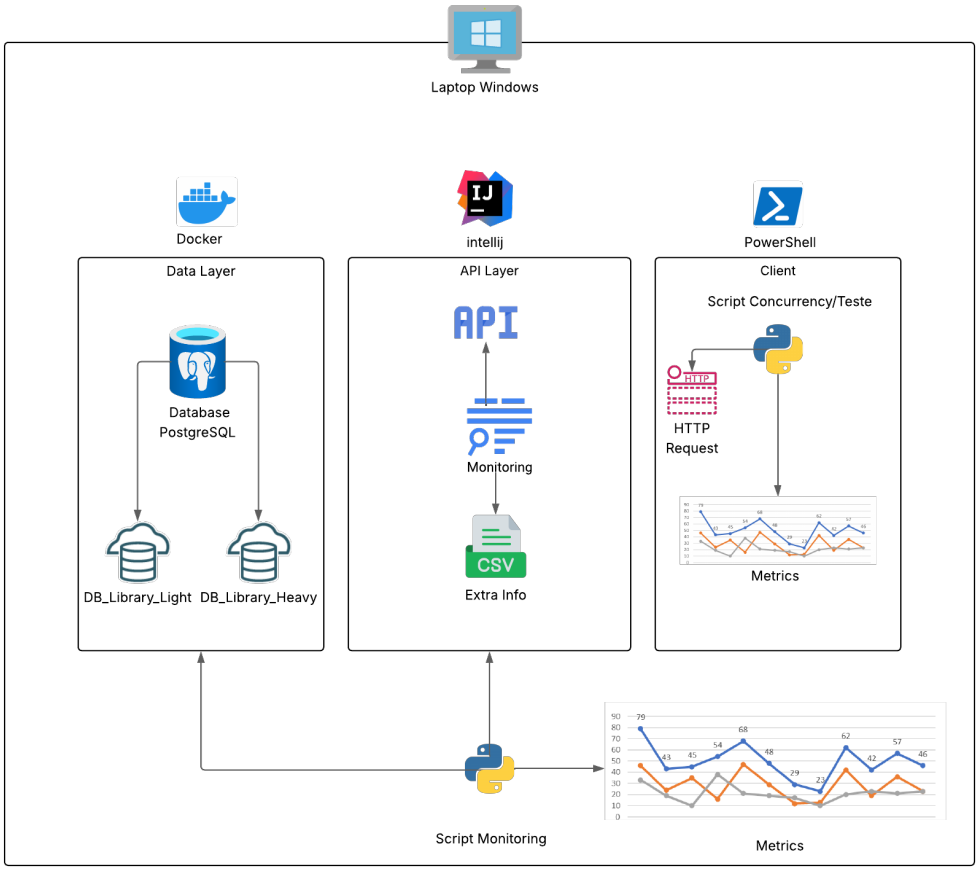

Figura 5.1: Representação esquemática da lógica e divisão do serviço.

5.2 Especificações de Hardwares

Nos testes realizados, e conforme descrito no Capítulo 5.1, todo o sistema foi implementado num único equipamento físico, com os recursos de hardware distribuídos pelas diferentes camadas. Os testes foram executados em duas equipamentos com hardwares diferentes.

Um dos sistemas usou um portátil LEGION 5 16IRX9-099, com as seguintes especificações:

- **Processador**: Intel Core i7-14650HX (16 *cores*, frequência base de 2.20 GHz, 24 *threads*)
- **Memória RAM**: 16 GB (15,7 GB utilizáveis) DDR5-SDRAM a 5600 MHz (1x16 GB)

- **Armazenamento**: SSD M.2 NVMe de 512 GB
- **Sistema Operativo**: Windows 11 Pro, versão N 24H2

A alocação dos recursos entre as diferentes camadas foi efetuada da seguinte forma:

- Para o container responsável pela execução do *Docker*, foram alocados 4 *cores* e 4 GB de memória RAM.
- Para a camada da API, foram atribuídos 4 *cores* e 8 GB de memória RAM, sendo esta memória igualmente utilizada para a configuração do *heap size*.

O segundo sistema usou um computador fixo com as seguintes caraterísticas:

- **Processador**: AMD Ryzen 7 5800X (8 *cores*, frequência base de 3.80 GHz, frequência máxima de até 4.70 GHz, 16 *threads*)
- **Memória RAM**: 32 GB DDR4-SDRAM a 2666 MHz
- **Armazenamento**: SSD de 1 TB
- **Sistema Operativo**: Windows 11 Pro, versão 24H2

A alocação dos recursos entre as diferentes camadas foi efetuada da seguinte forma:

- Para o container responsável pela execução do *Docker*, foram alocados 4 *cores* e 8 GB de memória RAM.
- Para a camada da API, foram atribuídos 4 *cores* e 16 GB de memória RAM, sendo esta memória igualmente utilizada para a configuração do *heap size*.

Relativamente à camada de clientes, não foram alocados recursos de hardware dedicados, uma vez que o seu impacto no desempenho global do sistema é reduzido no contexto deste estudo. Esta camada tem como principais responsabilidades a geração de *requests* para a API e a criação dos gráficos associados à análise de desempenho, em ambos os cenários de teste.

Esta configuração de alocação de recursos foi definida com o intuito de maximizar a semelhança com sistemas reais em ambiente de produção. Em cenários típicos, é comum que as bases de dados sejam executadas em máquinas virtuais ou containers com recursos limitados, de forma a otimizar

custos e permitir a coexistência com outros serviços críticos. Por outro lado, a camada da API tende a dispor de maior quantidade de memória, visto que processa simultaneamente múltiplos pedidos e depende fortemente da eficiência do seu *heap size* para reduzir a latência e aumentar a capacidade de resposta. Este padrão reflete práticas adotadas em ambientes de computação em nuvem, como AWS ou Azure, onde instâncias de bases de dados operam com recursos mais limitados do que servidores de aplicação escaláveis horizontalmente [26].

5.3 Questões técnicas

É relevante salientar algumas decisões metodológicas que impactam a interpretação dos resultados obtidos no estudo:

- **Não armazenamento das respostas da API**: Para evitar a sobreutilização de recursos do sistema e assegurar que os testes se concentrem exclusivamente no desempenho do processamento das requisições, as respostas geradas pela API não foram armazenadas. Esta abordagem reduz o consumo de memória e operações de I/O desnecessárias, mantendo o foco na análise de métricas relevantes.

- **Ausência de delay e interferência de rede**: Todo o sistema foi executado localmente num único equipamento, eliminando fatores externos como latência ou congestionamento de rede. Desta forma, as medições refletem exclusivamente o desempenho do processamento interno do sistema, sem interferência de elementos externos que não contribuem para o estudo.

Estas escolhas reforçam que o objetivo principal do experimento é avaliar a robustez e a capacidade de resposta do sistema sob diferentes níveis de carga, evitando que resultados sejam distorcidos por variáveis irrelevantes ao contexto específico da análise.

Adicionalmente, cumpre ressaltar que o presente estudo, como qualquer investigação experimental, pode estar sujeito a erros ou imprecisões metodológicas. Mesmo em condições de hardware idênticas, variações no ambiente de execução, no sistema operativo ou na própria sequência de testes podem ocasionar resultados divergentes. Dessa forma, os dados obtidos devem ser interpretados como referência indicativa do desempenho do sistema, servindo como base para análise e comparação, e não como determinação absoluta ou universal.

5.4 Divisão do Sistema

Nesta subsecção será apresentada a divisão do sistema nos seus principais módulos, com a devida descrição das decisões de projeto que nortearam a sua construção, bem como a lógica que sustenta a arquitetura proposta. Inicialmente, será abordada a modelagem e a implementação da base de dados, destacando-se as escolhas realizadas quanto à estrutura, à tecnologia utilizada. Em seguida, será discutido o desenvolvimento da API, enfatizando os critérios adotados na sua construção, os recursos disponibilizados, os objetivos de cada endpoint e os mecanismos implementados para o tratamento de falhas. Por fim, serão analisados os scripts de monitorização, que desempenham papel essencial na supervisão e no controle da aplicação. Nessa etapa, serão descritos os procedimentos de criação e execução desses scripts, bem como o funcionamento dos requests direcionados à API, salientando a forma como contribuem para a observabilidade, a manutenção e a estabilidade do sistema como um todo.

5.4.1 Backend

Esta secção descreve a conceção, estrutura e geração de dados para as bases de dados utilizadas nos testes desta tese. O objetivo é apresentar as decisões técnicas que levaram à criação de um ambiente controlado, mas com problemas intencionais, de forma a avaliar o impacto no desempenho e comportamento do sistema.

5.4.1.1 Esquema da Base de Dados

Neste projeto procurou-se reproduzir um sistema de bibliotecas, concebido propositadamente de forma incorreta, com o intuito de simular um cenário próximo de aplicações reais, mas que apresentasse sérios problemas de estrutura e implementação. O objetivo foi sobrecarregar o SGBD e a própria API com consultas mais pesadas do que o habitual, estabelecendo relações complexas e ineficientes.

Foram criadas duas bases de dados com o mesmo esquema, mas com quantidades de informação e carga distintas: DB_Library_Light e DB_Library _Heavy, que servirão de base para testes futuros.

Para maximizar o esforço computacional do SGBD, optou-se por abusar da utilização do tipo de dados `TEXT` no PostgreSQL, em conjunto com pesquisas do tipo `%ILIKE%`, reconhecidamente mais dispendiosas em termos de desempenho e tempo.

É igualmente sabido que a própria estrutura da base de dados contém campos de utilização questionável ou redundantes; tal foi feito propositadamente para cumprir o objetivo acima descrito. O esquema pode ser observado na figura 5.2.

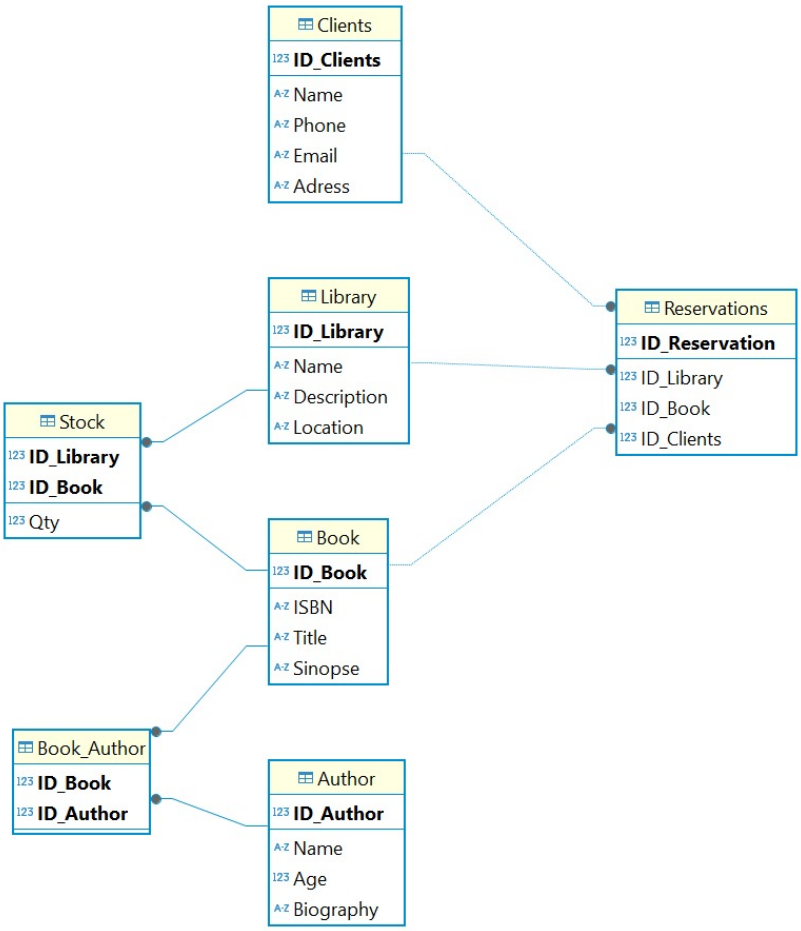

Figura 5.2: Este é o esquema da base de dados utilizada.

Como se pode observar na figura 5.2, o esquema apresenta algumas tabelas com estrutura próxima do ideal, enquanto outras revelam escolhas questionáveis.

A tabela Clients possui um desenho significativamente adequado, com `ID_Client` como chave primária e os campos `Name`, `Phone`, `Email` e `Address`, todos com pertinência e/ou sentido no contexto do sistema.

A tabela Reservations também se aproxima de um modelo coerente, contendo `ID_Reservation` como chave primária e as chaves estrangeiras `ID_Library`,

ID_Book e ID_Client, estabelecendo corretamente as relações entre reservas, bibliotecas, livros e clientes.

A tabela Library, por sua vez, apresenta ID_Library como chave primária e os campos Name, Description e Location. O campo Description, contudo, é de utilidade discutível, uma vez que não é util e/ou relevante para a maior parte das operações típicas de um sistema de bibliotecas.

A tabela Stock utiliza as chaves estrangeiras ID_Library e ID_Book como chave composta, juntamente com o campo Qty para indicar quantidade em stock. Apesar de funcional, sofre impacto das más práticas aplicadas nas tabelas relacionadas.

A tabela Book evidencia claramente algumas inconsistências de modelagem. Atualmente, utiliza ID_Book como chave primária, acompanhada dos campos ISBN, Title e Synopsis. Embora a escolha de ID_Book como chave primária proporcione consultas mais rápidas devido ao seu tipo inteiro, ela é questionável do ponto de vista conceitual, já que o ISBN constitui um identificador natural, único e universal para cada livro.

Uma abordagem híbrida poderia resolver este dilema: criar uma tabela intermediária que associe cada ISBN a um *ID_Book* artificial, utilizando índices para otimizar as pesquisas e joins. Dessa forma, mantém-se a integridade do identificador natural (ISBN), ao mesmo tempo que se garante performance elevada nas operações que envolvem relacionamentos com outras tabelas.

Além disso, o campo Synopsis, armazenado como TEXT, contribui significativamente para a sobrecarga de memória e para o aumento do tempo de execução das consultas, especialmente em operações de busca que envolvam múltiplas linhas.

A tabela Book_Author cumpre o papel de relação *many-to-many*, contendo apenas as chaves estrangeiras ID_Book e ID_Author.

Por fim, a tabela Author possui ID_Author como chave primária, e os campos Name, Age e Bibliography. Aqui observam-se dois problemas:

1. O campo Age não é adequado para bases bibliográficas, pois é um dado volátil que rapidamente se torna obsoleto e de relevância quase nula;

2. O campo Bibliography, também em TEXT, carrega informação extensa que raramente é necessária em consultas operacionais.

Em suma, embora algumas tabelas estejam próximas de um modelo adequado, a presença de campos desnecessários, escolhas discutíveis de chaves primárias e tipos de dados inadequados foi propositado e com o intuito de degradar a performance tornando-as consultas mais pesadas.

5.4.1.2 Geração de Dados

Como anteriormente referido, foram criadas duas bases de dados com o mesmo esquema, mas com volumes distintos de informação, de forma a simular diferentes cargas de trabalho e cenários de teste.

Para popular estas bases, foram desenvolvidos dois scripts em `Python` que utilizam a biblioteca `Faker` para gerar dados sintéticos. O `Faker` permite criar valores fictícios realistas para diversos tipos de campos, como nomes, endereços, textos longos, números de telefone, e-mails, entre outros. Cada execução do `Faker` produz dados diferentes, garantindo variedade e diversidade, mas mantendo a mesma estrutura e proporção de campos em ambas as versões da base.

Essa abordagem possibilita a simulação de cenários próximos de ambientes reais, permitindo testar a performance do sistema sob diferentes cargas: a versão *Light* contém dados de tamanho moderado, enquanto a versão *Heavy* inclui campos significativamente maiores, como textos extensos para nomes, descrições e sinopses de livros.

Ambos os scripts geram o mesmo número de campos e tipos de informação, porém com diferentes dimensões. Por exemplo, no campo *Name* da tabela *Library*, a versão *Heavy* pode gerar nomes com até 2000 caracteres, enquanto na versão *Light* o limite é de 100 caracteres. Essa diferença é ilustrada nos exemplos das Listagens 5.1 e 5.2, onde o parâmetro `max_nb_chars` define o tamanho máximo do texto gerado pelo `Faker`.

O aumento do tamanho dos campos impacta diretamente o tempo de execução das consultas, podendo causar *timeouts* e sobrecarga do sistema. Para manter a comparabilidade entre as bases, ambos os scripts não utilizam *seed*, de modo que cada execução produz valores diferentes, mas preserva a mesma estrutura e proporção de dados.

Excerto de Código 5.1: Exemplo de código em Python para gerar dados na versão *Heavy* da base de dados

```
def insert_libraries(n=500000):
    for _ in range(n):
        cursor.execute("""
            INSERT INTO "Library" ("Name", "Description", "
    Location")
            VALUES (%s, %s, %s)
        """, (
            fake.text(max_nb_chars=2000),
            fake.text(max_nb_chars=50000),
            fake.text(max_nb_chars=2000)
        ))
```

Excerto de Código 5.2: Exemplo de código em Python para gerar dados na versão *Light* da base de dados

```
def insert_libraries(n=500000):
    for _ in range(n):
        cursor.execute("""
␣␣␣␣␣␣␣␣␣␣␣␣INSERT␣INTO␣"Library"␣("Name",␣"Description",␣"
    Location")
␣␣␣␣␣␣␣␣␣␣␣␣VALUES␣(%s,␣%s,␣%s)
␣␣␣␣␣␣␣␣""", (
            fake.text(max_nb_chars=100),
            fake.text(max_nb_chars=250),
            fake.text(max_nb_chars=150)
        ))
```

5.4.1.3 Desafios

Dado que o objetivo inicial da base de dados era equilibrar realismo com performance reduzida, muitos campos foram intencionalmente definidos como extensos. Este design resultou em problemas de utilização de memória na API, especificamente na *heap size*, que passou de 4 GB previstos e atribuídos inicialmente para 8 GB durante o desenvolvimento desta tese.

Durante pesquisas, utilizando consultas do tipo `%ILIKE%`. grandes volumes de dados eram carregados na memória, excedendo a capacidade disponível. Aumentar ainda mais a memória poderia comprometer o estudo, pelo que foi necessário encontrar um *sweet spot* entre o número de parâmetros pesquisados e a quantidade de *RAM* utilizada.

Concluiu-se que o ideal seria limitar cada pesquisa a 4 palavras, utilizando consultas do tipo:

```
% ilike palavra1 palavra2 palavra3 palavra4 %
```

Esta limitação reduz substancialmente a carga de memória durante a execução das consultas. Embora fosse possível otimizar este comportamento diretamente na API, tal abordagem poderia comprometer os objetivos do estudo, que incluíam avaliar a performance sob condições de carga elevada.

A base de dados *Heavy* possui aproximadamente 50 GB de dados, ilustrando a magnitude do problema. Na versão *Light*, o mesmo efeito não ocorre tão exaustivamente devido ao menor volume de dados, mas manteve-se o limite de palavras por pesquisa para assegurar a comparabilidade das métricas de performance.

5.4.2 API - Base

5.4.2.1 Lógica

No âmbito desta tese foram desenvolvidas várias versões de uma mesma API, cada uma correspondendo à adaptação e/ou implementação de um mecanismo de cache distinto. Para efeitos de simplificação, neste capítulo será descrita apenas a API-Base e as suas principais características.

A aplicação foi construída com a *framework Spring Boot*, recorrendo ao *starter Spring Boot Starter Web*, que fornece suporte para o desenvolvimento de aplicações web e APIs RESTful através do Spring MVC. A aplicação utiliza o *Apache Tomcat* como servidor web embutido, responsável por receber os pedidos HTTP e devolver as respetivas respostas, não sendo necessária a instalação de um servidor externo.

O sistema foi projetado para responder a pedidos no contexto de um sistema de gestão de livrarias, focando-se exclusivamente em operações do tipo GET enquadrando-se com o objetivo do EPOS, destinadas à obtenção de informações específicas com base em parâmetros fornecidos pelo gestor. Desta forma, a API não permite inserções, alterações ou eliminações de dados, funcionando unicamente como um mecanismo de pesquisa e entrega de informação.

A estrutura da **API** organiza-se em *endpoints*, que permitem, por exemplo:

- Consultar livros por título;
- Consultar reservas por cliente;
- Consultar livrarias por nome.

Nas secções seguintes serão detalhados os principais *endpoints* implementados, os parâmetros aceites em cada requisição e o funcionamento geral do sistema.

5.4.2.2 Workflow

A API segue um fluxo estruturado em camadas, o que garante melhor manutenção, escalabilidade e reutilização de código. O processo inicia-se quando uma requisição HTTP é recebida através de um *endpoint*, definido num *Controller*, camada responsável pela comunicação entre o cliente e a aplicação.

O *Controller* atua como ponto de entrada da solicitação, interpretando os parâmetros recebidos e efetuando uma validação inicial dos dados. Em seguida, a requisição é encaminhada para a camada de Serviço.

A camada de Serviço interage com a camada de Dados, geralmente implementada segundo o padrão *Repository*, disponibilizado pelo Spring Data

JPA. Esta camada é responsável por gerar e executar as consultas necessárias, comunicando-se diretamente com a base de dados PostgreSQL. O resultado da consulta é processado pelo PostgreSQL e devolvido ao repositório sob a forma de registos estruturados. De seguida, o Spring converte esses registos em objetos Java (*Entities* ou *DTOs*), que regressam à camada de Serviço.

Os dados são então enviados de volta ao *Controller*, que prepara a resposta final. Antes do envio, a informação passa por um *parser*, responsável por formatar a saída de acordo com a preferência do cliente (JSON ou CSV). Finalmente, a resposta é devolvida através de um `HTTP Response`, concluindo o ciclo de requisição e resposta.

Este fluxo completo encontra-se representado no diagrama da figura 5.3.

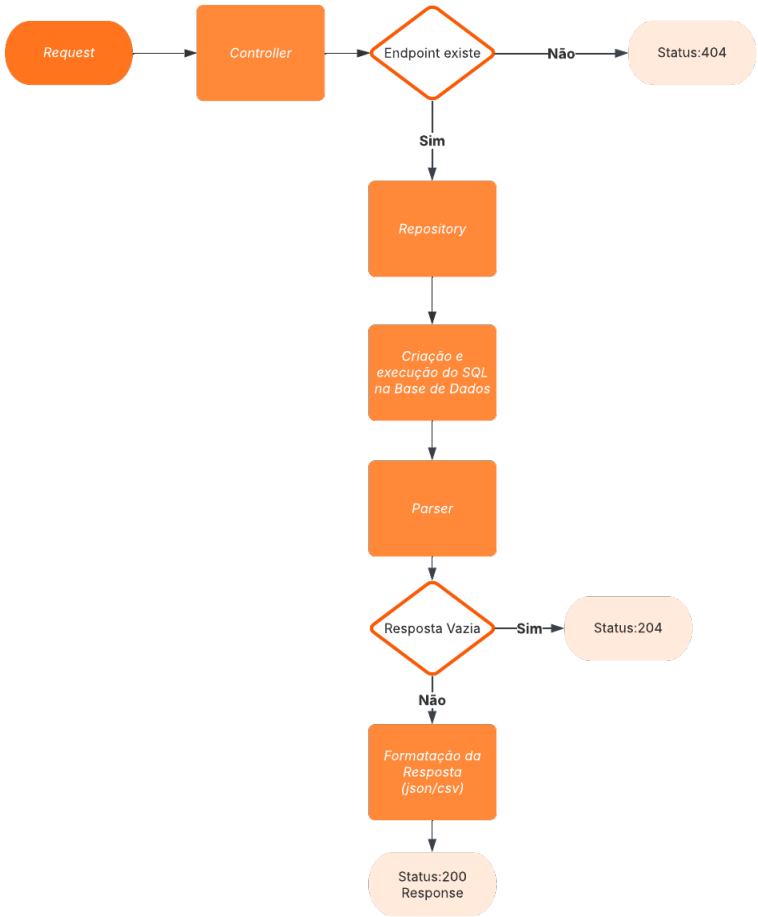

Figura 5.3: Diagrama do *workflow* de uma requisição à API.

5.5 Requests

5.5.1 Endpoints

O controller é responsável pela criação e gestão dos *endpoints*, que podem ser classificados em duas categorias principais: *endpoints sinples* e *endpoints compostos*.

Nos endpoints simples, as consultas geralmente envolvem apenas uma tabela da base de dados e retornam informações diretas, sem processamento adicional o exemplo de um endpoint simples é o responsável por obter informações de autores. O exemplo abaixo esta simplificado e foi reduzido para melhorar a compreensão do mesmo 5.3. O endpoint recebe um ou mais parâmetros do autor e procura na base de dados pelos autores com essas propriedades.

Excerto de Código 5.3: Codigo simplificado do endpoint.

```
@GetMapping("/Authors/{format}/")
public ResponseEntity<byte[]> getAuthors(
@PathVariable String format,
@RequestParam(required = false) Integer ID_Authors,
@RequestParam(required = false) String Name) throws ... {
    if (allNull(ID_Authors, Name))
        return badRequest(format, download, "Provide␣at␣
            least␣one␣parameter");

    List<Author> authors = authorRepository.
        getAuthorsParamether(ID_Authors, Name);

    if (authors.isEmpty())
        return emptyResponse(format, download);

    String result = parser.parseAuthors(format, authors);
    return returnResponse(format, download, result.getBytes(
        StandardCharsets.UTF_8));
}
```

O método `getAuthorsParameter` realiza a consulta diretamente na tabela `Author`, adicionando condições à query apenas para os parâmetros fornecidos e ordenando os resultados pelo identificador do autor como pode ser visto em 5.4:

Excerto de Código 5.4: Codigo para buscar a informação á Database

```
public List<Author> getAuthorsParameter(Integer ID_Author,
    String Name) {
    StringBuilder query = new StringBuilder("SELECT␣*␣FROM␣
        \"Author\"␣");
```

```
    Map<String, Object> params = new HashMap<>();
    List<String> conditions = new ArrayList<>();

    if (ID_Author != null) {
        conditions.add("\"ID_Author\" = :ID_Author");
        params.put("ID_Author", ID_Author);
    }
    if (Name != null && !Name.isBlank()) {
        conditions.add("\"Name\" ILIKE :Name");
        params.put("Name", "%" + Name + "%");
    }

    if (!conditions.isEmpty()) {
        query.append("WHERE ").append(String.join(" AND ",
            conditions));
    }

    query.append(" ORDER BY \"ID_Author\" ASC;");
    return namedParameterJdbcTemplate.query(query.toString()
        , params, (rs, rowNum) -> mapRow(rs));
}
```

Por ultimo a resposta ao mesmo depende do tipo de formato que é construido no parser e pode ser visto no exemplo do parser em 5.6

5.6 Parser

Após a resposta da base de dados para o java esta resposta passa por um parser que dependendo do formato escolhido pelo utilizador converte o objeto Java para esse formato. Diferentes endpoints passam por parsers diferentes e com caraterísticas diferentes mas todos retornam a resposta no formato string. O excerto de código seguinte é um exemplo para o endpoint combinado de Books.

```
public String parseBooks(String format, List<Book> books,
    String AuthorName,Integer Author_id) throws
    JsonProcessingException {
        switch(format.toLowerCase()){
            case "json":
                ObjectMapper mapper = new ObjectMapper();
                List<Map<String, Object>> wrappedBooks = new
                    ArrayList<>();
                for (Book book : books) {
                    Map<String, Object> bookContent = new
                        HashMap<>();
                    bookContent.put("ID_Book", book.getid())
                        ;
```

```
                    bookContent.put("ISBN", book.getIsbn());
                    bookContent.put("Sinopse", book.
                        getSinopse());
                    bookContent.put("Author_Name",AuthorName
                        );
                    bookContent.put("Author_ID",Author_id);
                    Map<String, Object> bookMap = new
                        HashMap<>();
                    bookMap.put("Title:" + book.getTitle(),
                        bookContent);
                    wrappedBooks.add(bookMap);
                }

                res= mapper.writerWithDefaultPrettyPrinter()
                    .writeValueAsString(wrappedBooks);
                break;
            case "csv":
                StringBuilder csvBuilder = new StringBuilder
                    ();
                csvBuilder.append("ID␣Book,ISBN,Author_Name,
                    Author_ID,Title,Sinopse").append(System.
                    lineSeparator());
                for (Book book : books) {
                    csvBuilder.append(book.getID_Book()).
                        append(",");
                    csvBuilder.append("\"").append(book.
                        getIsbn().replace("\"", "\"\"")).
                        append("\",");
                    csvBuilder.append("\"").append(
                        AuthorName.replace("\"", "\"\"")).
                        append("\",");
                    csvBuilder.append("\"").append(Author_id
                        ).append("\",");
                    csvBuilder.append("\"").append(book.
                        getTitle().replace("\"", "\"\"")).
                        append("\",");
                    csvBuilder.append("\"").append(book.
                        getSinopse().replace("\"", "\"\"")).
                        append("\"\n");
                }
                res = csvBuilder.toString();
                break;
            default:
                res="Wrong␣format";
                break;
        }
        return res;
    }
```

Este código gera a resposta seguinte

```
 {
"Title:Within career real term.": {
  "ISBN": "978-1-4286-5248-4",
  "Author_Name": "Hot increase four detail different
     nature. Full several fast act run.",
  "ID_Book": 10,
  "Author_ID": 537498,
  "Sinopse": "What all past nearly news account. Affect
     third fine speech kitchen. Maybe ten vote fear
     skill. Identify garden stage."
}
```

Nos endpoints compostos, há relações entre as tabelas, permitindo retornar informações mais completas e significativas para o cliente. Por exemplo os de reservas (*Reservations*), nos quais diversas tabelas se relacionam. Uma pesquisa por reservas pode envolver informações de bibliotecas, livros, autores, clientes e das próprias reservas, tornando este tipo de endpoint mais pesado computacionalmente, mas essencial para fornecer dados completos ao cliente.

5.7 Parâmetros

Como referenciado acima diferentes endpoints permitem parâmetros diferentes. Nos endpoints simples podemos usar um parâmetro por chamada ou a combinação direta de um ou mais que pertencem a mesma categoria. Apesar de não muito usado nos testes realizados é uma caraterística que pode ser abordada no futuro. De modo geral os endpoints aceitam sempre ID's e nomes bem como caraterísticas únicas de cada tabela fazendo com que o tipo de entradas sejam (int e/ou string). Nos endpoints complexos existe apenas uma entrada correta, esperando ou um int ou uma string dependendo do endpoint. Caso os parâmetros sejam nulos e/ou inválidos a API retorna erro 404. Se foram válidos mas não existir correspondência dá status 204. Se tiverem correspondência da status 200 e por ultimo temos o código 500 quando a Database da timeout.

5.8 Coleta de Dados

Como mencionado anteriormente a própria API gera dados para monitorizar a performance da mesma. Estes dados são capturados durante a execução da mesma em cada endpoint e avaliam parâmetros como tempo para resposta

da database tempo para parser tempo usado de cpu e memoria ram. Estes dados sao escritos num csv para gerar graficos e representações visuais dos mesmos.

```
dia,endpoint,params,db_time_nanos,parser_time_nanos,
    total_time_nanos,bytes_sent,status,memoria_b,cpu_nanos
2025-10-18,/Authors/{format}/,ID_Authors=1 Name=null Age=
    null Bibliograhy=null
    ,31833401,11565600,465059401,367,200,4194304,62500000
2025-10-18,/Authors/{format}/,ID_Authors=1 Name=null Age=
    null Bibliograhy=null
    ,0,744301,53263501,367,200,0,31250000
```

5.8.1 Erros

5.8.2 Limitações de String

Strings com mais de 65.535 caracteres causaram exceções em alguns endpoints devido à limitação do tipo `TEXT` e uso incorreto de buffers.

5.8.3 Heap Size (8 GB)

Durante testes com grande volume de dados, a JVM atingiu o limite de heap de 8 GB, resultando em:

- Lentidão
- OutOfMemoryError
- Reinício de instância

5.9 Chamadas à API

A solução desenvolvida em *Python* não é utilizada como um simples *script* isolado, mas sim funciona como uma **Biblioteca**, permitindo a geração de chamadas a diferentes *endpoints* da API. Dessa forma, pode ser reutilizada por outros módulos e *scripts* implementados nesta tese, sendo invocada sempre que uma chamada à API deve ser construída.

Para a simulação dos parâmetros transmitidos, em cada requisição, emprega-se a biblioteca *Faker*, responsável pela geração de dados pseudoaleatórios de forma reprodutível. A reprodutibilidade é garantida pela definição de uma *seed* fixa, sendo adotado, neste trabalho, o valor `2100`. Ressalta-se ainda que

a escolha desse valor foi realizada de maneira arbitrária, não estando associada a qualquer critério técnico, estatístico ou estudo prévio. O objetivo é unicamente assegurar que diferentes execuções resultem nos mesmos valores, permitindo, assim, a comparação consistente dos resultados obtidos.

5.9.1 Definição dos Endpoints

A lista de endpoints disponíveis é definida de forma explícita, associando a cada caminho (*path*) um identificador de tipo (*id*) e a função responsável pela geração dos parâmetros correspondentes:

```
self.endpoints_list = [
{"path": "/Books/", "id": "Simple", "fn": self._Books},
{"path": "/Authors/", "id": "Simple", "fn": self._Authors},
{"path": "/Library/", "id": "Simple", "fn": self._Library},
{"path": "/Clients/", "id": "Simple", "fn": self._Clients},
{"path": "/Reservations/", "id": "Simple", "fn": self.
    _Reservations},
{"path": "/Books/", "id": "Book", "fn": self.
    _Books_AuthorName},
{"path": "/Books/", "id": "Book", "fn": self._Books_AuthorID
    },
{"path": "/Books/", "id": "Book", "fn": self._Books_BookName
    },
{"path": "/Books/", "id": "Book", "fn": self._Books_BookID},
{"path": "/Reservations/", "id": "Reservation", "fn": self.
    _Reservations_ID_Reservation},
{"path": "/Reservations/", "id": "Reservation", "fn": self.
    _Reservations_ClientName},
{"path": "/Reservations/", "id": "Reservation", "fn": self.
    _Reservations_LibraryName},
{"path": "/Reservations/", "id": "Reservation", "fn": self.
    _Reservations_BookTitle},
{"path": "/Stocks/", "id": "Stock", "fn": self.
    _Stocks_BookTitle},
{"path": "/Stocks/", "id": "Stock", "fn": self.
    _Stocks_Library},
]
```

A seleção de um endpoint ocorre de forma pseudoaleatória, baseada na seed.

5.9.2 Geração de Parâmetros

Uma vez escolhido o endpoint, são gerados os parâmetros de entrada. A geração combina funções da biblioteca *Faker* com valores inteiros pseudoalea-

tórios, de modo a representar dados realistas, mas controlados para reprodutibilidade:

```
def generate_params(self, endpoint_data, index):
local_random = random.Random(self.seed + index)
local_faker = Faker()
local_faker.seed_instance(self.seed + index)

'''
all_params = endpoint_data["fn"](local_faker, local_random)
chosen_key = local_random.choice(list(all_params.keys()))
chosen_value = all_params[chosen_key]
chosen_format = local_random.choice(FORMATS)

return {
    chosen_key: chosen_value,
    "format": chosen_format
}, chosen_key, endpoint_data["id"], endpoint_data["path"]
'''
```

5.9.3 Construção da URL Final

A partir dos parâmetros gerados, a URL correspondente é construída com base no tipo de endpoint, respeitando diferentes convenções de formatação:

```
def generate_output_string(self, index):
endpoint_data = self.get_random_endpoint(index)
params, main_key, endpoint_id, path = self.generate_params(
   endpoint_data, index)
format_type = params["format"]
main_value = params[main_key]

'''
if endpoint_id == "Simple":
    return f"{path}{format_type}/?{main_key}={main_value}"
elif endpoint_id == "Book":
    return f"{path}{format_type}/{main_key}/{main_value}/"
elif endpoint_id == "Reservation":
    return f"{path}{format_type}/{main_key}/{main_value}/
       Info"
elif endpoint_id == "Stock":
    return f"{path}{format_type}/{main_key}/{main_value}"
'''
```

5.9.4 Funções de Geração de Dados

Cada endpoint possui uma função específica, responsável por produzir os dados que alimentam as chamadas. Essas funções utilizam tanto números aleatórios quanto valores sintéticos realistas (nomes, títulos, descrições) fornecidos pelo *Faker*:

```
def _Books(self, f, r):
return {"ID_Book": r.randint(1,1500000),
"ISBN": f.isbn13(),
"Title": f.sentence(nb_words=4).rstrip("."),
"Sinopse": f.paragraph(nb_sentences=1).rstrip(".")}

def _Authors(self, f, r):
return {"ID_Authors": r.randint(1,100000),
"Name": f.sentence(nb_words=4).rstrip("."),
"age": r.randint(0, 100),
"Bibliography": f.sentence(nb_words=4).rstrip(".")}
```

De forma análoga, são definidos métodos para bibliotecas, clientes, reservas e stock.

5.10 Avaliação da API : Concorrência

Para avaliar o desempenho da API, foram realizados diversos testes de performance com o objetivo de identificar níveis de concorrência adequados. Essa abordagem permitiu evitar execuções redundantes em cenários nos quais não haveria variação significativa nos resultados.

O termo concorrência refere-se ao número de chamadas simultâneas feitas à API. Quanto maior esse número, maior é a demanda sobre os recursos do sistema, podendo resultar em degradação de desempenho.

Para determinar os pontos ideais de carga foram realizadas dez mil chamadas à API, distribuídas em diferentes níveis de concorrência. Em cada configuração eram executadas mil chamadas idênticas, mantendo-se um número fixo de execuções paralelas por exemplo: 10 chamadas simultâneas. Sempre que uma chamada era concluída, outra era iniciada imediatamente, garantindo que o nível de concorrência permanecesse constante durante todo o teste.

Além disso, a cada nova execução, o nível de concorrência era incrementado em 10, variando de 10 até 100 chamadas simultâneas, em intervalos regulares.

Os testes de concorrência também servem para simular diferentes níveis de demanda no servidor, refletindo cenários reais nos quais o sistema recebe

requisições com maior ou menor frequência.

5.10.1 Script de Testes de Concorrência

O script utilizado nos testes operava da seguinte forma: inicialmente, gerava mil chamadas pseudo-aleatórias, que eram executadas respeitando o limite de concorrência definido. Sempre que uma vaga ficava disponível, uma nova chamada era iniciada.

Paralelamente, o script monitorava métricas de desempenho e utilização de recursos, incluindo:

1. Número de chamadas bem-sucedidas e com erro;
2. Tempo médio de resposta;
3. Uso de CPU (Docker e API);
4. Consumo de memória RAM (Docker e API);
5. Operações de leitura e escrita em cada recurso monitorado (Docker e API).

Ao final de cada execução, os dados coletados eram consolidados e transformados em gráficos, apresentados para análise subsequente.

O script simplificado utilizado pode ser consultado no repositório bem como a criação dos gráficos.

5.10.2 Gráficos

Após a execução dos testes experimentais, foram produzidos gráficos que sintetizam os resultados obtidos, estruturados em diferentes categorias de análise, conforme segue:

- **API**: métricas referentes à utilização de CPU, memória ram (RAM) e operações de entrada/saída (I/O);
- **Docker**: métricas de utilização de CPU, memória RAM e intensidade das operações de I/O no contexto de contêinerização;
- **Status Response**: distribuição entre respostas bem-sucedidas e respostas com falha (código 500) ao longo da execução;
- **Média Response Time**: tempo médio de resposta para requisições com retorno 200/204 e 500, discriminado por nível de concorrência.

Essas representações gráficas configuram ferramentas essenciais para a análise aprofundada do comportamento da API e da camada de persistência sob diferentes níveis de carga. A partir delas, torna-se possível identificar padrões de desempenho ótimo, zonas críticas de saturação e potenciais gargalos associados à alocação de recursos computacionais ou à degradação na taxa de sucesso das requisições.

No cenário específico aqui investigado, a *API-BASE* foi adotada como grupo de controle e referência para a avaliação dos diferentes níveis de concorrência. A execução ocorreu em ambiente *database heavy*, no qual se verificou a ocorrência de *timeout* da base de dados após 60 segundos. Esse fator impõe restrições adicionais à análise, uma vez que determinadas variações nos resultados podem ser atribuídas diretamente a essa limitação. Por essa razão, destaca-se a necessidade de interpretar os dados de forma holística, considerando o conjunto completo das métricas e evitando inferências baseadas em indicadores isolados.

Todos os gráficos produzidos encontram-se organizados e disponíveis no meu repositório [27]. Os dados recolhidos dos mesmos estão sintetizados e resumidos na tabela seguinte 5.1.

5.10.3 Avaliação : Resultados e Conclusões

Tabela 5.1: Resultados de Desempenho por Concorrência

	API				Base de dados				Respostas	
Conc.	CPU méd.	CPU máx.	RAM méd.	RAM máx.	CPU méd.	CPU máx.	RAM méd.	RAM máx.	Tempo médio	Taxa de acerto (%)
10	2.5	55.5	5.45	8.13	97.3	109.6	3.09	3.20	17.28	91.0
20	2.6	94.5	4.77	7.69	99.1	108.5	3.13	3.20	39.45	91.0
30	2.9	72.2	4.40	8.06	96.1	106.5	3.11	3.21	61.89	89.5
40	2.9	69.5	4.23	8.25	95.0	107.6	3.11	3.21	83.17	88.1
50	1.5	52.7	3.38	6.74	95.2	107.7	3.14	3.23	69.39	60.8
60	1.4	41.8	4.98	7.30	95.7	109.2	3.14	3.21	76.10	47.0
70	1.2	80.0	3.71	4.29	94.2	108.3	3.18	3.22	75.92	35.5
80	1.2	42.5	3.59	4.16	94.7	106.0	3.18	3.21	74.27	31.5
90	2.2	41.0	3.69	7.74	94.1	106.0	3.16	3.23	70.44	26.8
100	0.8	28.5	3.97	4.09	94.8	106.5	3.17	3.20	74.12	27.8

A análise dos resultados mostra que, até à concorrência 40, a taxa de erro se mantém próxima de 10%, indicando um funcionamento estável do sistema. A partir da concorrência 50 observa-se uma degradação acentuada, com a taxa de erro a atingir cerca de 40%, ultrapassando 50% na concorrência 60 e crescendo progressivamente até aproximadamente 72% na concorrência 100.

O consumo de CPU da API apresenta um comportamento não linear. Em níveis mais baixos de concorrência (10 a 40), a utilização média situa-se entre 2,5% e 3%, refletindo um maior número de respostas bem-sucedidas. Os picos mais elevados de CPU ocorrem nas concorrências 20, 30, 40 e 70. A

partir da concorrência 50, a média de utilização reduz-se significativamente, atingindo valores inferiores a 1% na concorrência 100, consequência direta do aumento de *timeouts* e da diminuição de respostas processadas.

O consumo de memória RAM da API varia de forma mais irregular, dependendo não apenas do nível de concorrência, mas também do tamanho e da quantidade das respostas. Até à concorrência 40, as médias situam-se em torno de 4–5 GB, com picos superiores a 8 GB. A partir da concorrência 50 observa-se uma redução geral, embora ocorram picos pontuais mesmo em níveis elevados, evidenciando a influência do tipo de carga processada.

Na base de dados, tanto o uso de CPU como de RAM mantêm-se relativamente constantes ao longo dos diferentes níveis de concorrência. Este comportamento, aparentemente contra intuitivo, é explicado pela dinâmica dos *timeouts*. Entre as concorrências 10 e 40, o tempo médio de resposta cresce rapidamente. A partir da concorrência 50 ocorre uma inflexão: o tempo médio das respostas bem-sucedidas diminui, enquanto a taxa de *timeouts* aumenta significativamente.

Quando múltiplos *timeouts* expiram em simultâneo, várias requisições são libertadas ao mesmo tempo, gerando picos momentâneos de carga na base de dados. Este fenómeno resulta numa redução do tempo total de execução do teste de cerca de 36 minutos na concorrência 10 para aproximadamente 13 minutos na concorrência 100 à custa de um elevado número de falhas. Assim, a aparente melhoria no tempo total não representa ganho de desempenho, mas sim colapso operacional.

Em síntese, a concorrência 50 marca um ponto crítico a partir do qual a estabilidade do sistema se degrada significativamente, com os *timeouts* a dominarem o comportamento da API e a influenciarem diretamente a utilização de recursos e os tempos de resposta.

Com base nos resultados apresentados, foram selecionados os níveis de concorrência 30, 50 e 70 para a avaliação da implementação de cache, por representarem três regimes distintos de funcionamento do sistema:

Concorrência 30 - Representa um cenário estável, com taxa de erro próxima de 10%. Serve como referência de desempenho normal, antes do ponto de degradação.

Concorrência 50 - Corresponde ao ponto de inflexão do sistema, onde a taxa de erro aumenta significativamente e os *timeouts* começam a impactar de forma relevante o desempenho global.

Concorrência 70 - Caracteriza um cenário de elevada carga e forte degradação, adequado para avaliar o comportamento do sistema sob stress intenso e identificar possíveis efeitos de saturação.

Capítulo

6

Implementação das Caches

6.1 Introdução

Neste capítulo, será apresentada a forma como foram implementadas as diferentes caches, destacando as suas principais características, a lógica subjacente ao processo de desenvolvimento e o raciocínio técnico que guiou as decisões de implementação. Serão ainda discutidas as dificuldades encontradas, as estratégias utilizadas para as ultrapassar e as limitações inerentes a cada abordagem.

Importa salientar que este capítulo se foca exclusivamente na descrição e análise das implementações desenvolvidas, bem como nas opções técnicas associadas. Os resultados obtidos a partir dessas implementações, assim como a respetiva análise comparativa serão abordados no capítulo seguinte.

6.2 Pensamentos

Como referido anteriormente, a cache é utilizada para armazenar, de forma mais rápida e eficiente, informação que é constantemente acedida 2.1. Para que este mecanismo funcione corretamente, é fundamental analisar detalhadamente a arquitetura e a estrutura da aplicação, de modo a identificar os pontos de maior interesse para a implementação da cache nomeadamente, as consultas mais frequentes ou demoradas e distinguir esses pontos daqueles de menor relevância.

É igualmente essencial garantir um equilíbrio adequado entre o sistema e o mecanismo de cache. Tal como referido em 2.1, uma má gestão ou configuração da cache pode resultar no efeito inverso ao pretendido, conduzindo a uma degradação do desempenho global do sistema. Este fenómeno é particu-

larmente evidente no caso da database heavy, nos quais a grande quantidade de dados e a frequência elevada de operações de inserção e remoção (insert/-delete) na cache causam maior sobrecarga do que benefícios efetivos.

Outro aspeto relevante nesta implementação é o facto de a cache partilhar recursos com o restante sistema, que opera de forma monolítica. Assim, existe uma limitação física de 16 GB de memória RAM, a qual é distribuída por todos os componentes da arquitetura nomeadamente o Docker, a API, o sistema principal e o módulo de cliente, que inclui os scripts responsáveis pela geração dos pedidos (requests).

Superadas estas restrições, a lógica de funcionamento da cache mantém-se relativamente simples. A cada pedido efetuado, é verificado se já existe na cache um registo correspondente a uma chamada anterior à API com os mesmos parâmetros. Caso exista, a resposta é obtida diretamente da cache, evitando o acesso à base de dados. Caso contrário, procede-se à consulta da base de dados original e o resultado é então armazenado na cache. Importa referir que o que é guardado não é o resultado final da operação, mas sim o objeto correspondente, permitindo a sua reutilização em futuras chamadas. Após a inserção na cache, o pedido prossegue o seu curso normal e, dependendo do endpoint em questão, podem ocorrer diferentes comportamentos, que serão detalhados nas secções seguintes.

6.2.0.1 Endpoints Simples

Como exemplo prático da lógica de cache aplicada a um endpoint simples, pode-se considerar o endpoint `/Authors/format/`. Este endpoint suporta diversos parâmetros opcionais que permitem filtrar os resultados, tais como `ID_Authors`, `Name`, `Age` e `Bibliography`. Importa salientar que a base de dados foi deliberadamente construída para ser sobrecarregada e não otimizada, motivo pelo qual alguns destes parâmetros podem parecer redundantes ou pouco relevantes no contexto de uma aplicação de produção eficiente.

Quando um pedido é enviado para a API através deste endpoint, o sistema analisa os parâmetros incluídos podendo ser um ou vários e, com base neles, constrói uma chave única de identificação na cache. Essa chave é gerada de forma determinística, combinando os nomes e os valores dos parâmetros recebidos, o que garante que pedidos idênticos originam sempre a mesma chave.

Por exemplo, numa chamada do tipo `?Name="John"&Age=67`, a chave resultante seria `Name=John&Age=67`. Esta abordagem permite ao sistema reconhecer rapidamente pedidos repetidos e associá-los à mesma entrada na cache. Após a geração da chave, é verificada a sua existência na cache, originando dois possíveis cenários:

- **Se a chave existir:** o sistema devolve diretamente o resultado armazenado na cache, evitando o acesso à base de dados reduzindo significativamente o tempo de resposta.

- **Se a chave não existir:** é realizada uma chamada à fonte original dos dados a base de dados sendo o conjunto de resultados obtido armazenado na cache e associado à chave gerada. Este resultado é então utilizado no processamento do pedido atual e permanece disponível para futuras consultas com os mesmos parâmetros.

6.2.0.2 Endpoints Complexos

Nos endpoints de maior complexidade, a estratégia de cache é adaptada à estrutura e ao objetivo de cada caso. Um exemplo representativo é o endpoint `/Books/{format}/AuthorName/{Author_Name}/`, em que a chave de cache corresponde diretamente ao nome do autor. Esta abordagem permite partilhar entradas de cache entre endpoints distintos. Por exemplo, uma pesquisa no endpoint `/Authors/{format}/` com o parâmetro `Name=John` gera a mesma chave, possibilitando a reutilização da entrada já existente. Após a validação inicial, o sistema consulta a base de dados para obter os identificadores dos livros associados ao autor. Em seguida, verifica quais desses livros já se encontram na cache: os disponíveis são recuperados diretamente, enquanto os restantes são obtidos da base de dados e adicionados à cache. Desta forma, este endpoint utiliza de forma coordenada duas caches uma dedicada a autores e outra a livros, otimizando o desempenho e garantindo a consistência dos dados.

6.2.0.3 Endpoints de Resposta Direta

Por fim, destaca-se o endpoint `/Stocks/format/Book/BookTitle`, cuja implementação segue uma lógica simplificada. Neste caso, é armazenada na cache a resposta final do endpoint, utilizando como chave o parâmetro do título do livro e o formato do pedido. Esta estratégia elimina a necessidade de executar novamente o processo de parsing e processamento dos dados em chamadas subsequentes, resultando numa resposta mais rápida e eficiente.

6.3 Implementação

Na próxima secção é demonstrado a implementação de cada cache, caraterísticas únicas, problemas encontrados e limitações das mesmas.

6.3.1 EHCACHE

6.3.1.1 Informação

Para a implementação do sistema de cache foi utilizada a biblioteca **Ehcache** na sua versão **3.10.8**, a mais recente disponível no momento da realização desta tese. O Ehcache é uma solução de cache de alto desempenho e fácil integração em aplicações Java, apresentando uma configuração simples e um conjunto robusto de funcionalidades. Entre as suas principais características destacam-se:

- **Flexibilidade no armazenamento:** permite escolher o local onde a cache é armazenada em memória (heap), fora da memória da JVM (off-heap) e em disco, apesar do local onde é armazenada, ela pode armazenar em disco de forma permanente se assim quiser.
- **Persistência:** possibilidade de manter o estado da cache mesmo após a reinicialização da aplicação, evitando recomputações desnecessárias.
- **Gestão de expiração:** suporta múltiplos algoritmos de *Time-to-Live* (TTL) e *Time-to-Idle* (TTI), permitindo controlar a validade temporal dos dados armazenados.
- **Gestão por tamanho:** o Ehcache pode gerir o armazenamento com base no tamanho ocupado (por exemplo, em MB), em vez de se limitar ao número de entradas.

Uma limitação relevante observada é a ausência de suporte nativo a uma *cache miss explícita* isto é, uma entrada de chave existente mas sem valor associado (útil para indicar que uma pesquisa anterior não retornou resultados). Para contornar esta limitação, é possível inserir um objeto vazio (por exemplo, uma `ArrayList` vazia) e validar a sua presença para evitar novas consultas redundantes à base de dados.

6.3.1.2 Implementação

Para utilizar as caches temos de implmentar e isso faz-se da seguinte forma. Primeiramente, é necessário adicionar a dependência no `build.gradle` (compilador usado no desenvolvimento desta tese.):

```
implementation 'org.ehcache:ehcache:3.10.8'
```

De seguida, cria-se uma classe de configuração responsável por instanciar o `CacheManager` e definir os parâmetros da cache, nomeadamente o nome

"BookSearCache"esta foi usada para armazenar os livros que retornam da procura deles nos endpoints. Ela aceita como chave uma string e devolve uma ArrayList, guarda ate 250mb fora da heap e remove os dados ao fim de 15 minutos.Este e so um exemplo de muitas configurações possiveis. Outro detalhe e quando esta cache esta cheia e entra outra entrada ela usa por padrão LRU (Least Recently Used) este padrão remove o *item* menos recentemente acessado. Mas possui ainda mais 2 opções LFU (Least Frequently Used) este remove o *item* menos frequentemente acessado e FIFO (First In, First Out) que remove o *item* que entrou primeiro no cache.

```
@Bean
public CacheManager cacheManager() {
CacheManager cacheManager = CacheManagerBuilder.
   newCacheManagerBuilder()
.withCache("BookSearchCache",
CacheConfigurationBuilder.newCacheConfigurationBuilder(
String.class, ArrayList.class,
ResourcePoolsBuilder.newResourcePoolsBuilder()
.offheap(250, MemoryUnit.MB)
)
.withExpiry(Expirations.timeToLiveExpiration(
Duration.of(15, TimeUnit.MINUTES)))
)
.build(true);
return cacheManager;
}
```

Depois de configurada a cache, temos de criar uma instância da mesma para usar no nosso controller.

```
@Autowired
private CacheManager cacheManager;

private Cache<String, ArrayList<Book>> getSearchBookCache()
   {
    return (Cache<String, ArrayList<Book>>)

(Cache<?, ?>) cacheManager.getCache(
"BookSearchCache", String.class, ArrayList.class);
}
```

A utilização prática deste mecanismo pode ser observada, por exemplo, no endpoint "/Books/format/AuthorName/Author_Name/"onde aplicamos a cache.

```
//Criação da chave
String Key = "Name="+Author_Name;
```

```
//Criação da cache
Cache<String,ArrayList<Author>> AuthorSearchCache =
    getsearchAuthorCache();
List <Author> authors= new ArrayList<>();

//Verificação na Cache
if (AuthorSearchCache.get(Key)!= null){
    if (AuthorSearchCache.get(Key).isEmpty()){
        makeLog(endpoint,params,startTime,startMem,startCpu
            ,0,0,204,0);
        return emptyResponse(format, download);
    }else{
        authors.addAll(AuthorSearchCache.get(Key));
    }
}

//Sem registos na cache
if (authors.isEmpty()){
    long dbTime2= System.nanoTime();
    authors=authorRepository.getAuthorsParamether(null,
        Author_Name,null,null);
    dbTime += duration(dbTime2);
    if (authors.isEmpty()) {
        //Cache miss case
        AuthorSearchCache.put(Key,new ArrayList<>());
        makeLog(endpoint,params,startTime,startMem,startCpu,
            dbTime,0,204,0);
        return emptyResponse(format, download);
    }else{
        //Adicionar na Cache
        AuthorSearchCache.put(Key,new ArrayList<>(authors));
    }
}
```

Neste exemplo, gera-se uma chave única através da junção do parâmetro "Name="e o campo pesquisado (Relembro que este formato e para reutilizar esta cache noutro endpoint). Em seguida, tenta-se recuperar os resultados a partir da cache. Caso a cache contenha um valor associado à chave gerada que (seja diferente de null), verifica-se esse valor não está vazio. Caso esse valor seja vazio, ou seja, a lista vazia, é tratada noutro caso para simular um miss respondemos logo à request. Caso contrario, os dados são adicionados à lista de Autores que será devolvida.

Por outro lado, se a cache não contiver qualquer resultado, procede-se à consulta direta na base de dados. Após a recuperação dos dados, os resultados são armazenados na cache, garantindo que pesquisas subsequentes com os mesmos parâmetros possam ser atendidas e respondidas através da cache.

6.3.2 Guava

6.3.2.1 Informação

Para a implementação do sistema de cache foi utilizada a biblioteca **Guava Cache**, na versão **33.2.1**, a mais recente disponível no momento da realização desta tese.

As principais funcionalidades disponibilizadas pela biblioteca incluem:

- **Evicção e Expiração** Permite definir um tamanho máximo para o cache, removendo automaticamente entradas quando o limite é atingido. Adicionalmente, suporta políticas de expiração baseadas no tempo, como `expireAfterWrite` e `expireAfterAccess`.
- **Carregamento Automático** Oferece suporte ao carregamento automático de valores ausentes através de *cache loaders*, reduzindo a necessidade de verificações manuais.
- **Política de Substituição** Utiliza políticas de substituição baseadas principalmente em recência de acesso (aproximação ao algoritmo *Least Recently Used – LRU*), sendo uma abordagem simples e eficaz para cenários de cache em memória.

Comparativamente com a biblioteca *Ehcache*, a Guava Cache não suporta o armazenamento persistente dos dados, fazendo com que todo o conteúdo do cache seja perdido a cada reinício da aplicação.

6.3.2.2 Implementação

Para utilizar a cache com Guava, é necessário adicionar a dependência correspondente no ficheiro `build.gradle`, utilizado no desenvolvimento desta tese:

```
implementation 'com.google.guava:guava:33.2.1-jre'
```

De seguida, é criada uma classe de configuração responsável por instanciar e gerir as diferentes caches da aplicação. O exemplo seguinte apresenta um método genérico para a criação de caches parametrizáveis:

```
public <K, V> Cache<K, V> createCache(String name, long
    maxSize, long expireMinutes) {
Cache<K, V> cache = CacheBuilder.newBuilder()
.maximumSize(maxSize)
.expireAfterWrite(expireMinutes, TimeUnit.MINUTES)
.recordStats()
.build();
```

```
caches.put(name, cache);
return cache;
}
```

A seguir apresenta-se um exemplo da utilização do mecanismo de cache numa operação de consulta:

```
String key = cacheKeyClient(ID_Client, Name, Phone, Email,
    Address);

List<Client> clients = searchClientCache.getIfPresent(key);
if (clients == null) {
long startDb = System.nanoTime();
clients = clientRepository.getClientsParameter(
ID_Client, Name, Phone, Email, Address
);
dbTime += duration(startDb);
searchClientCache.put(key, clients);
}
```

Neste exemplo, é gerada uma chave única a partir dos parâmetros fornecidos no endpoint, permitindo a reutilização do cache em diferentes chamadas. Inicialmente, tenta-se obter o resultado diretamente da cache. Caso não exista um valor associado à chave, é efetuada uma consulta à base de dados. O resultado dessa consulta é então armazenado na cache, reduzindo o custo de chamadas futuras com os mesmos parâmetros.

6.3.3 Caffeine

6.3.3.1 Informação

Para a implementação do sistema de cache foi utilizada a biblioteca **Caffeine** na sua versão **3.1.8**, a mais recente disponível no momento da realização desta tese. Caffeine é uma biblioteca Java de cache em memória de alto desempenho, inspirada no API do Guava, mas com melhorias significativas em desempenho e funcionalidades:

- **Evicção e Expiração** Permite definir um tamanho máximo para o cache, removendo automaticamente entradas quando o limite é atingido. Também suporta políticas de expiração baseadas no tempo,
- **Carregamento Automático** Oferece suporte para o carregamento automático de valores ausentes.
- **Política de Admissão e Evicção Inteligente (W-TinyLFU)** O Caffeine utiliza o algoritmo *Window TinyLFU*, que combina políticas de recência e

frequência para decidir quais entradas manter ou descartar. Essa abordagem equilibra a inserção de novos elementos e a retenção dos mais frequentemente acessados.

- **Estatísticas e Métricas** O sistema fornece métricas detalhadas de desempenho, incluindo taxa de acertos (*hit rate*), falhas (*miss rate*), tempo médio de carregamento e número de evicções.

Comparativamente a cache anterior esta não suporta guardar os dados de forma permanente, fazendo com que a cada reinicio a cache seja esvaziada.

6.3.3.2 Implementação

Para utilizar as caches temos de implmentar e isso faz-se da seguinte forma. Primeiramente, é necessário adicionar a dependência no `build.gradle` (compilador usado no desenvolvimento desta tese.):

```
implementation 'com.github.ben-manes.caffeine:caffeine:3.1.8
    '
```

De seguida, cria-se uma classe de configuração responsável por instanciar o `CacheManager` e definir os parâmetros da cache, nomeadamente o nome "AuthorSearchCache". Esta foi usada para armazenas os autores que retornam da procura deles nos endpoints. Ela aceita como chave uma string e devolve uma ArrayList. Guarda ate 300 entradas e remove os dados ao fim de 10 minutos. Este e so um exemplo de muitas configurações possíveis.

```
public static final Cache<String, List<Author>>
    AuthorSearchCache = Caffeine.newBuilder()
            .expireAfterWrite(10, TimeUnit.MINUTES)
            .maximumSize(300)
            .build();
```

A implementação do mecanismo de cache pode ser visto a seguir.

```
        String key= cacheKeyAuhtors(ID_Authors,Name,age,
            Bibliography);
        List<Author> authors = CacheManager.
            AuthorSearchCache.getIfPresent(key);

        if (authors == null) {
            long startDb1 = System.nanoTime();
            authors = authorRepository.getAuthorsParamether(
                ID_Authors, Name, age, Bibliography);
            dbTime = duration(startDb1);
            CacheManager.AuthorSearchCache.put(key,authors);
        }
```

```
        if (authors.isEmpty()){
            makeLog(endpoint,params,startTime,startMem,
                startCpu,dbTime,0,204,0);
            return emptyResponse(format, download);
        }

        StringBuilder response = new StringBuilder();
        if (format.equalsIgnoreCase("csv")) {
            response.append("ID␣Author,Name,Age,Bibliography
                ").append(System.lineSeparator());
        }

        long parseTime=System.nanoTime();
        response.append(parser.parseAuthors(format, authors)
            );
        parseTime=duration(parseTime);

        String result = response.toString();
        makeLog(endpoint,params,startTime,startMem,startCpu,
            dbTime,parseTime,200,result.length());
        return returnResponse(format, download, result.
            getBytes(StandardCharsets.UTF_8));
    }
}
```

Neste exemplo, gera-se uma chave única através da junção dos parâmetros pedidos no endpoint (Relembro que este formato e para se reusar esta cache noutro endpoint). Em seguida, tenta-se recuperar os resultados a partir da cache. Caso a cache não contenha um valor associado à chave gerada (seja null), vamos a base de dados para responder a esta chamada. È adicionado então a lista authors o valor retornado pela cache ou pela consulta esta é adicionada á cache.

6.3.4 Ignite

6.3.4.1 Informação

O *Apache Ignite* distingue-se dos mecanismos de cache tradicionais por atuar como uma base de dados em memória (*in-memory data grid*) e não apenas como um sistema de cache volátil. Essa arquitetura híbrida permite o armazenamento e o processamento de dados em tempo real, mas, por outro lado, implica algumas limitações práticas na sua utilização como mecanismo de cache puro.

Por ser uma solução relativamente recente e mais orientada a bases de dados distribuídas do que a caches convencionais, o Ignite carece de certas

funcionalidades típicas desses mecanismos, tais como estratégias automáticas de expulsão (*eviction policies*) e mecanismos nativos de invalidação.

Uma das principais restrições identificadas é o limite de 64 KB por entrada na base de dados, o que impõe constrangimentos significativos à sua implementação em sistemas que demandam armazenamento de objetos de maior dimensão. Além disso, o Ignite utiliza uma interface baseada em SQL para manipulação dos dados, exigindo que o programador implemente manualmente todas as operações fundamentais (inserção, remoção, atualização, etc.).

Cada tipo de cache deve ser representado por uma tabela distinta no sistema, podendo-se, contudo, adotar uma abordagem genérica. Todavia, essa generalização tende a introduzir complexidades adicionais na manutenção e no desempenho do sistema.

6.3.4.2 Implementação

A implementação do Ignite requer a execução de um sistema de nó responsável pela gestão da cache, que deve operar num *cluster*. Para o contexto deste projeto, foi configurado apenas um nó local dentro do cluster, permitindo a execução e o teste do sistema em ambiente isolado (configurações alternativas são discutidas em seções posteriores).

A seguir, apresenta-se um excerto de código que demonstra a criação de uma classe responsável pela gestão do mecanismo de cache, de forma semelhante às demais soluções utilizadas neste estudo:

```
public CacheManager() throws Exception {
    this.client = IgniteClient.builder()
            .addresses("127.0.0.1:10800") // ajustar
               conforme o endereço do cluster
            .build();
}

@PostConstruct
public void initTables() {
    String[] tables = {
            "stockBookCache",
            "BookCache",
            "AuthorCache",
            "ClientCache",
            "BookSearchCache",
            "AuthorSearchCache",
            "ClientSearchCache",
            "BookSCache",
            "ReservationCache"
    };
```

```
    for (String table : tables) {
        try {
            client.sql().execute(null, "DROP␣TABLE␣IF␣EXISTS
                ␣" + table);

            String createSql = "CREATE␣TABLE␣IF␣NOT␣EXISTS␣"
                + table +
                    "␣(id␣VARCHAR␣PRIMARY␣KEY,␣data␣
                        VARBINARY)";
            client.sql().execute(null, createSql);

            System.out.println("Tabela␣inicializada:␣" +
                table);

        } catch (Exception e) {
            System.err.println("Erro␣ao␣inicializar␣a␣tabela
                ␣" + table + ":␣" + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Inicialmente, estabelece-se a conexão com o servidor de cache. Em seguida, executam-se instruções SQL para remover eventuais tabelas antigas e criar novas instâncias, assegurando um estado inicial limpo. Embora o Ignite suporte armazenamento persistente em disco, a configuração padrão utiliza armazenamento em memória (*RAM*).

Nesta classe, foram ainda desenvolvidos métodos auxiliares destinados à inserção e recuperação de dados na cache, possibilitando a abstração das operações de acesso aos dados.

A seguir, apresenta-se um exemplo de método de integração com a *API*, no qual a cache é utilizada para otimizar a recuperação de informações sobre reservas de clientes:

```
@GetMapping("/Reservations/{format}/client/{ClientName}/Info
    ")
public ResponseEntity<?> getReservationInfoByClientName(
        @PathVariable String format,
        @PathVariable String ClientName)
        throws IOException, ClassNotFoundException {

    String Key = "Name=" + ClientName;
    List<Client> clients = (List<Client>) cacheManager.
        getIfPresent("ClientSearchCache", Key);

    if (clients == null) {
        long dbTime2 = System.nanoTime();
```

```
        clients = clientRepository.getClientsParamether(null
            , ClientName, null, null, null);
        dbTime += duration(dbTime2);
        cacheManager.put("ClientSearchCache", Key, ArrayList
            .class);
    }

    if (clients.isEmpty()) {
        makeLog(endpoint, params, startTime, startMem,
            startCpu, dbTime, 0, 204, 0);
        return emptyResponse(format, false);
    }
}
```

Este método exemplifica a estratégia de leitura condicional na cache: inicialmente, tenta-se obter os dados do cliente diretamente da cache (*cache hit*); caso a informação não esteja presente, a consulta é realizada na base de dados, sendo posteriormente armazenada no Ignite para acesso futuro e tendo suporte a (*cache miss*).

Essa abordagem permite reduzir a latência das operações e otimizar o desempenho geral da aplicação, embora a ausência de mecanismos automáticos de invalidação e a limitação de tamanho imponham desafios adicionais à sua utilização em ambientes de produção.

6.3.5 Considerações entre elas e dificuldades encontradas

A análise comparativa entre os mecanismos de cache implementados *Caffeine*, *Guava*, *Ehcache* e *Apache Ignite* permite identificar semelhanças, limitações e vantagens específicas de cada solução, bem como as principais dificuldades encontradas durante o processo de integração e utilização prática no contexto do projeto.

6.3.5.1 Caffeine e Guava

As bibliotecas Caffeine e Guava Cache apresentam APIs e filosofias de uso semelhantes, uma vez que o Caffeine surgiu como sucessor direto do Guava Cache, com foco em maior desempenho e eficiência. Contudo, apesar da semelhança externa, as duas diferem significativamente na sua arquitetura interna e nos algoritmos de substituição utilizados.

Ambas armazenam os dados exclusivamente na heap memory da JVM, não oferecendo suporte nativo a armazenamento persistente ou off-heap.

O Caffeine diferencia-se por empregar o algoritmo de substituição W-Tiny LFU (Windowed Tiny Least Frequently Used), que combina critérios de recência e frequência para maximizar a taxa de acertos do cache. Quando o limite

de capacidade é atingido, são removidas preferencialmente as entradas menos relevantes segundo esses critérios. O parâmetro `maximumSize` refere-se ao número máximo de entradas armazenadas, e não ao consumo de memória em bytes.

Adicionalmente, o Caffeine suporta políticas de tamanho baseadas em pesos, permitindo um controle mais refinado do uso de memória. A biblioteca não permite o armazenamento de valores nulos e incentiva o uso do método `Cache.get(key, mappingFunction)`, que é atômico e seguro em ambientes concorrentes.

A Guava Cache, por sua vez, utiliza políticas de substituição mais simples, baseadas principalmente em recência (LRU), e oferece uma interface de uso semelhante.

6.3.5.2 Ehcache

O *Ehcache* destaca-se por ser uma das soluções mais tradicionais e simples de configurar. A sua adoção requer apenas a importação da biblioteca, a inicialização no método principal da aplicação e, opcionalmente, a criação de um arquivo de configuração `ehcache.xml`. Contudo, observou-se que, em versões mais recentes, o uso deste arquivo apresenta incompatibilidades, o que impõe a configuração programática como alternativa.

O Ehcache suporta armazenamento em diferentes níveis *heap*, *off-heap* e disco permitindo a definição de políticas de persistência, de substituição (*LRU*, *LFU*, *FIFO*) e de tamanho (por número de entradas ou por consumo de memória). Essa flexibilidade, embora vantajosa, aumenta a complexidade de configuração, especialmente em aplicações que utilizam múltiplas caches com diferentes características.

Entre as dificuldades mais recorrentes estão o tratamento de valores nulos (que devem ser armazenados de forma explícita para evitar falhas de leitura), a necessidade de configurações específicas para chaves compostas e a impossibilidade de armazenar múltiplos objetos distintos sob a mesma chave. O uso de XML, apesar de familiar, mostrou-se problemático em termos de compatibilidade e manutenção.

Outro desafio significativo é a coerência entre diferentes caches, especialmente quando estas não suportam chaves complexas de maneira uniforme. Além disso, o uso de armazenamento off-heap pode causar inconsistências se a versão do JDK não for totalmente compatível com o gerenciador de memória nativo utilizado.

Por fim, a gestão de múltiplas instâncias de cache dentro da mesma aplicação aumentou consideravelmente a complexidade da lógica, demandando

controle rigoroso das chaves e políticas de invalidação, sob o risco de corromper a integridade dos dados armazenados.

6.3.5.3 Ignite

O *Apache Ignite* distingue-se das demais soluções por não ser apenas uma biblioteca de cache local, mas uma plataforma de dados em memória distribuída (*in-memory data grid*). Essa característica o torna mais adequado para ambientes que exigem escalabilidade horizontal e compartilhamento de dados entre múltiplos nós de um cluster.

Ao contrário das outras ferramentas, o Ignite requer a inicialização de um servidor dedicado de cache, que gerencia a comunicação e a consistência entre os nós. Essa abordagem, embora poderosa, torna sua configuração consideravelmente mais complexa, envolvendo parâmetros avançados de rede, replicação, persistência e particionamento de dados.

O Ignite combina conceitos de cache e base de dadps, permitindo consultas via SQL diretamente sobre os dados em cache. Essa versatilidade, porém, exige maior esforço de implementação, especialmente para mapear objetos de aplicação em tabelas SQL e garantir coerência transacional.

Durante o desenvolvimento, observou-se que o Ignite impõe restrições como o limite de 64 KB por entrada, ausência de mecanismos automáticos de expulsão e a necessidade de manipulação manual de operações básicas de inserção e remoção. Tais fatores aumentam a complexidade do código e a propensão a erros de configuração.

6.3.5.4 Síntese Comparativa e Dificuldades Gerais

Em termos gerais, *Caffeine* e *Guava* destacam-se pela leveza e facilidade de integração, sendo ideais para aplicações de médio porte que requerem apenas cache local em memória. Já o *Ehcache* oferece um equilíbrio entre simplicidade e persistência, sendo adequado quando há necessidade de armazenamento em disco ou off-heap. O *Ignite*, por sua vez, apresenta-se como a solução mais robusta e escalável, porém também a mais complexa de implementar e manter.

As principais dificuldades encontradas incluem:

- A necessidade de lidar manualmente com valores nulos e listas vazias em todas as implementações;

- A gestão de múltiplas caches e configurações distintas, o que aumenta a complexidade e o risco de inconsistências;

- Problemas de compatibilidade de versões (especialmente com Ehcache e Ignite);
- A dificuldade em manter coerência entre caches quando se utilizam chaves compostas;
- A inexistência de métodos automáticos de inserção condicional ou invalidação em Caffeine e Guava;
- A curva de aprendizagem elevada e o número significativo de parâmetros necessários para configurar corretamente o Ignite.

Em suma, a escolha do mecanismo de cache mais adequado depende fortemente do contexto e dos requisitos de escalabilidade e persistência. Enquanto Caffeine, Guava e Ehcache são soluções eficazes para caches locais e aplicações monolíticas, o Apache Ignite destaca-se em cenários distribuídos e de alto desempenho, onde a complexidade de configuração é compensada pelos ganhos de escalabilidade horizontal e consistência de dados em ambientes de cluster.

Capítulo

7

Comparação

7.1 Introdução

Neste capítulo, serão apresentados e analisados os dados recolhidos durante a execução dos testes. O principal objetivo é comparar o comportamento dos diferentes mecanismos de cache sob diversas condições de carga e níveis de concorrência, conforme descrito na Secção1.4. Além disso, será incluída uma análise comparativa com a aplicação base, ou seja, a versão do sistema que não utiliza qualquer tipo de mecanismo de cache, de modo a evidenciar o impacto real da introdução de caches no desempenho global.

A análise incidirá sobre um conjunto de métricas de desempenho fundamentais, nomeadamente a utilização de CPU, memória e disco, bem como o tempo médio de resposta. Adicionalmente, serão considerados indicadores de qualidade, como o número de respostas corretas e incorretas obtidas em cada cenário experimental.

Com base nestas métricas, pretende-se avaliar o impacto de cada mecanismo de cache na eficiência global do sistema, identificando as suas vantagens e limitações. Esta comparação, incluindo a aplicação base sem cache, permitirá compreender de forma mais aprofundada como diferentes estratégias de cache influenciam o desempenho e a escalabilidade das aplicações sob condições variáveis de concorrência e carga.

Nas secções seguintes, serão apresentados os resultados obtidos, seguidos da respetiva análise e discussão, de forma a sustentar as conclusões finais deste estudo.

7.2 Representação Gráfica

Após a implementação das APIs e a conclusão dos testes, os dados de desempenho recolhidos foram analisados. Nessa etapa, surgiram diversos desafios relacionados à forma de representar os resultados. Muitas métricas apresentaram pequenas diferenças absolutas ou baixa variância, mas as suas variações relativas (baseadas em percentagem) eram mais expressivas e, portanto, exigiram um tratamento bastante cuidadoso. Outra dificuldade decorreu da natureza intrinsecamente multi-variada do conjunto de dados: múltiplas dimensões precisaram ser projetadas num espaço bidimensional para permitir uma interpretação visual significativa. Para lidar com isso, técnicas comummente utilizadas em *Machine learning* foram aplicadas para melhorar a clareza das representações gráficas.

Apesar dos dados estarem estruturados em componentes separados a sua interpretação deve ser realizada de forma conjunta. Examinar isoladamente uma única métrica, campo ou gráfico é insuficiente, porque o comportamento do sistema resulta do efeito combinado de várias condições experimentais. Cada cenário produz uma resposta distinta moldada pelo seu contexto específico e nenhuma configuração universal apresenta o melhor desempenho em todas as situações. Consequentemente, interpretar os resultados exige bastante cautela e requer uma leitura atenta combinada de todos os elementos disponíveis.

Informações adicionais foram incluídas, como o nível de concorrência e o estado de operação: cold, correspondente ao sistema com o cache vazio e em processo de inicialização, e *operational*, correspondente ao sistema com o cache cheio e em estado estável. Todas essas variações são comparáveis à API base. Entretanto, esta última não foi representada separadamente nos modos *cold* e *operational*, pois não foram observadas diferenças relevantes entre eles.

Foram criados dois tipos de gráficos para avaliar parâmetros diferentes.

O primeiro tipo de gráfico (gráfico de barras), presente em 7.3 e 7.7 mostra a taxa de acertos por tipo de cache e nível de concorrência, permitindo a comparação da eficiência entre os modos cold e operacional. As barras representam o valor absoluto de acertos obtidos, enquanto o final de cada barra indica a variação percentual em relação à API base.

O eixo y corresponde ao nível de concorrência. É importante observar que valores mais altos ou mais baixos de concorrência não são intrinsecamente melhores ou piores, representando apenas diferentes configurações experimentais. Por sua vez, o eixo x mostra o número absoluto de acertos com valores maiores indicando melhor desempenho.

O segundo tipo de gráfico (gráfico de dispersão) presente em 7.4 e 7.8

apresenta, em duas dimensões, a relação entre: (i) o tempo total de execução em minutos ou segundos (eixo x, em que valores menores são preferíveis) e (ii) o número de acertos (eixo y, em que valores maiores são desejáveis).

7.2.1 Heavy Database

Em seguida são representados os dois tipos de gráficos descritos acima para a base de dados *Heavy* e no hardware do cenário 1 (portátil).

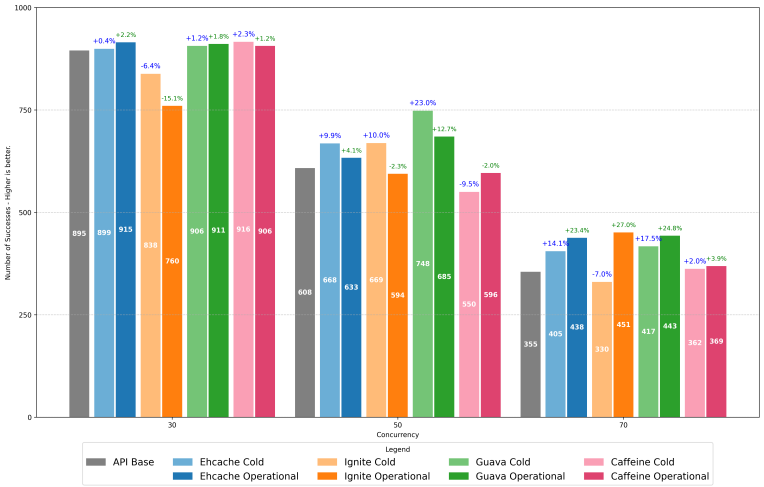

Figura 7.1: Gráfico de barras da base de dados Heavy cenário 1.

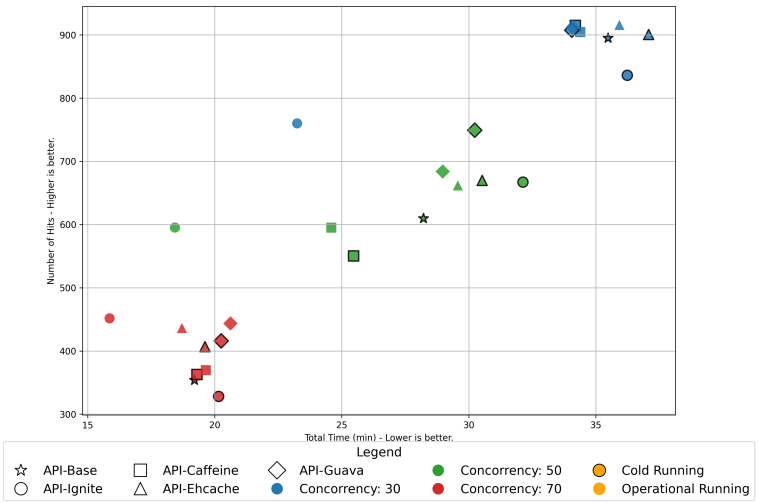

Figura 7.2: Gráfico de dispersão da base de dados Heavy cenário 1.

Analisando os dois gráficos apresentados, pode-se concluir que, de forma geral, a utilização de mecanismos de cache contribuiu para o aumento da taxa de acerto (hit rate) da API. No entanto, observam-se algumas exceções, particularmente nos cenários de baixa e média concorrência, sendo o Ignite um exemplo relevante.

Ao analisar o gráfico de dispersão, destaca-se que, apesar de apresentar uma redução na taxa de acerto, o Ignite é a solução que aparenta obter o maior ganho em termos de tempo de execução quando comparado com os restantes concorrentes. Verifica-se ainda que, em baixa concorrência, os valores encontram-se fortemente concentrados, evidenciando comportamentos próximos entre as diferentes soluções de cache: algumas aumentam a taxa de acerto à custa de um maior tempo de execução, enquanto outras conseguem simultaneamente reduzir o tempo de execução e aumentar a taxa de acerto.

No cenário de média concorrência observa-se a maior dispersão dos resultados. Ainda assim, todas as soluções analisadas demonstram melhorias em pelo menos um dos parâmetros avaliados tempo de execução ou taxa de acerto indicando aparentemente um impacto globalmente positivo da utilização de cache neste nível de carga.

Por fim, em alta concorrência, todas as soluções apresentam uma melhoria consistente na taxa de acerto, embora acompanhada por um aumento do tempo de resposta. Importa salientar que este tempo corresponde ao tempo total de execução e não ao tempo médio por chamada, o qual poderá ser calculado posteriormente caso se revele pertinente para uma análise mais detalhada.

Em seguida são representados os dois tipos de gráficos descritos acima para a base de dados *Heavy* e no hardware do cenário 2 (computador fixo).

Nestes outros casos, observa-se que, apesar das semelhanças entre os gráficos de dispersão nos dois cenários nomeadamente uma distribuição semelhante dos dados e um comportamento consistente entre as diferentes caches , várias destas apresentaram resultados inferiores quando comparados com o desempenho normal da API-Base. Este efeito é particularmente notório na cache Ignite, devido às limitações de hardware do cenário 2, nomeadamente o menor número de *cores* (8). Ainda assim, a relação entre as caches e a variação dos seus resultados mantém-se coerente em ambos os gráficos.

No gráfico de barras, verifica-se que, apesar de os valores absolutos de acertos e falhas serem completamente distintos, a estrutura global permanece semelhante, o que corrobora o desempenho relativo observado. Destaca-se, contudo, o caso da cache Caffeine no ambiente de concorrência 70, que evidenciou uma perda de desempenho significativa, provavelmente associada ao trabalho adicional de alocação e remoção imposto pelo algoritmo, aliado às limitações de hardware.

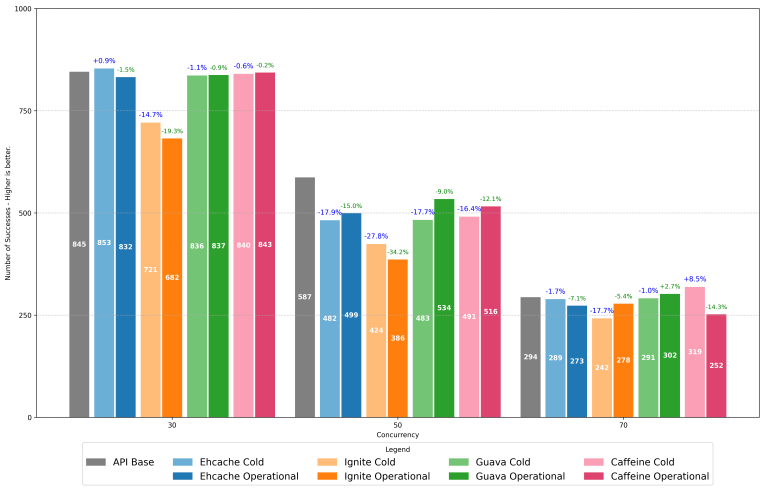

Figura 7.3: Gráfico de barras da base de dados Heavy cenário 2.

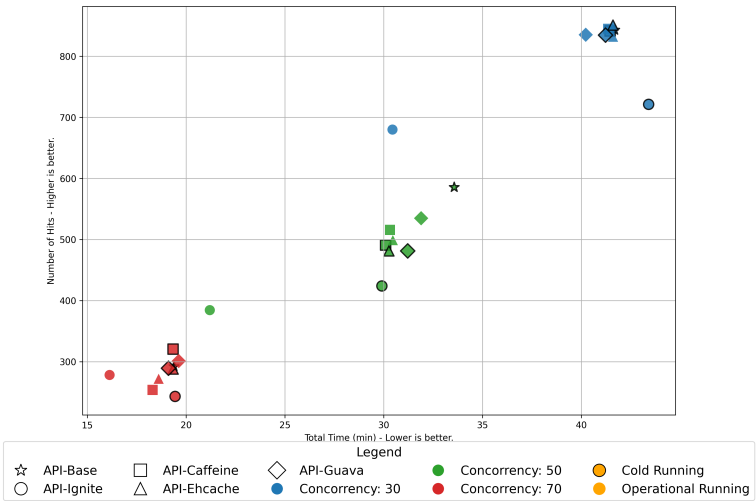

Figura 7.4: Gráfico de dispersão da base de dados Heavy cenário 2.

7.2.2 Light Database

No caso da base de dados *Light*, verifica-se que apenas a cache Ignite apresenta uma diminuição da taxa de acerto, enquanto as restantes soluções mantêm ou melhoram este indicador.

No gráfico de dispersão, observa-se que, no modo cold, os valores encontram-se fortemente concentrados, agrupando-se no canto superior direito do gráfico, o que indica comportamentos semelhantes entre as diferentes soluções neste cenário. Em contraste, no modo operational, os resultados tornam-se

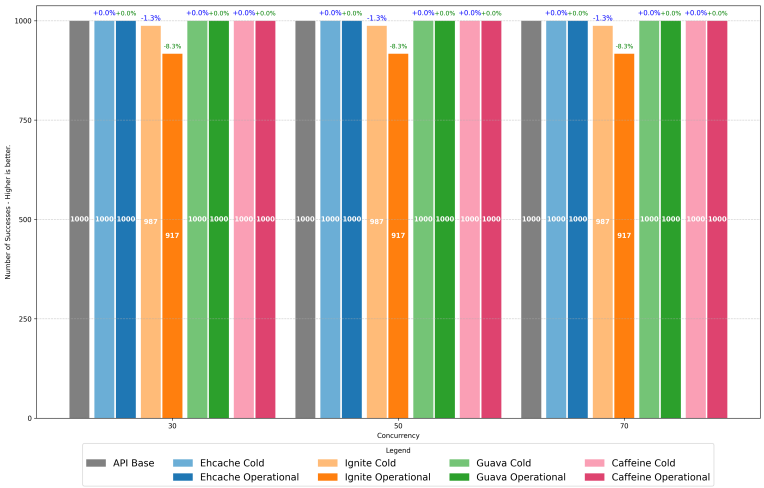

Figura 7.5: Gráfico de barras da base de dados Light no cenário 1.

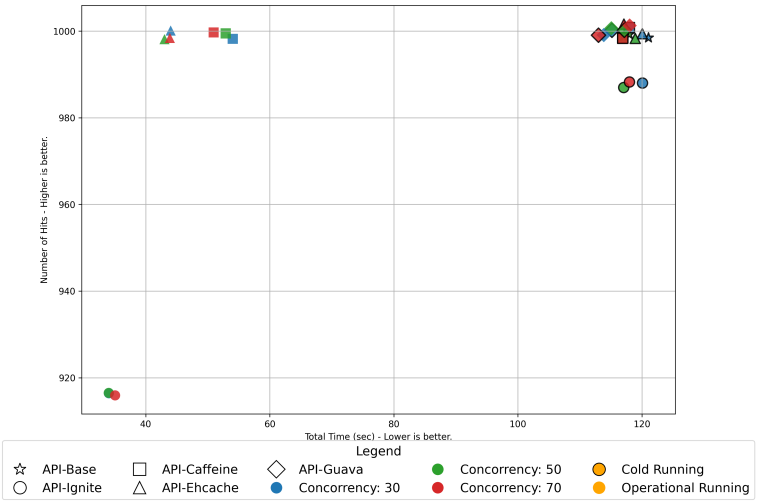

Figura 7.6: Gráfico de dispersão da base de dados Light no cenário 1.

mais dispersos, evidenciando diferenças mais claras entre as abordagens avaliadas. Neste contexto, a solução que demonstrou melhor desempenho foi a Ehcache, ao conseguir reduzir o tempo de execução sem perda na taxa de acerto, sendo seguida de perto pela Caffeine. Um caso particular é o da Guava, cuja performance se manteve essencialmente neutra, não apresentando melhorias significativas neste cenário.

Ignorando o fator da taxa de acerto, a Ignite destaca-se como a solução com maior melhoria em termos de tempo de execução. Adicionalmente, os

seus resultados apresentam uma elevada concentração, indicando que, neste cenário, o nível de concorrência teve um impacto reduzido no desempenho, uma vez que os valores se mantiveram praticamente sobrepostos.

No contexto do cenário 2 foram gerados os seguintes gráficos:

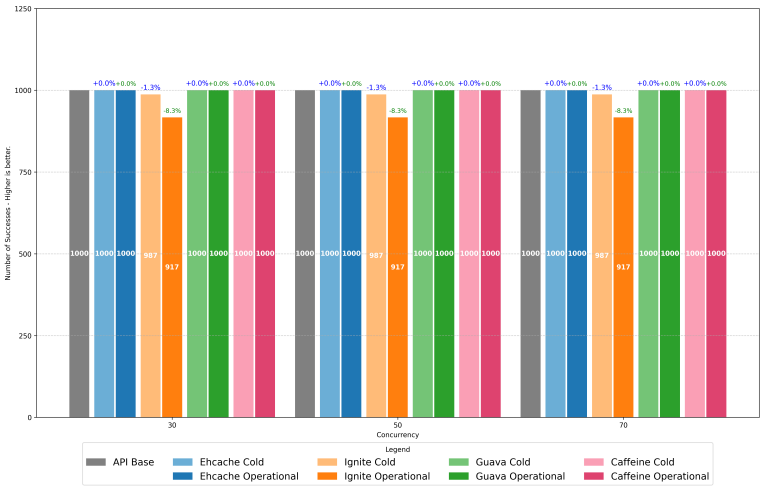

Figura 7.7: Gráfico de barras da base de dados Light no cenário 1.

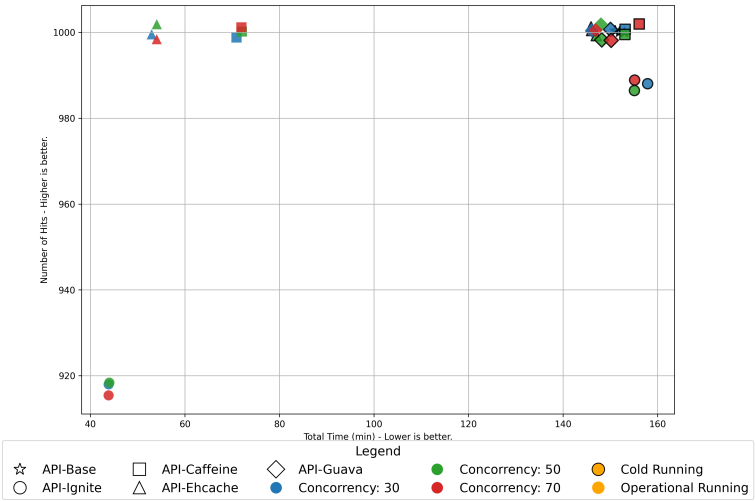

Figura 7.8: Gráfico de dispersão da base de dados Light no cenário 1.

As mesmas conclusões observadas no cenário heavy podem também ser verificadas no cenário light, evidenciando de forma ainda mais clara a existência de uma relação consistente entre ambos. Os valores obtidos mostram-se condizentes entre os dois cenários e casos analisados.

A partir daqui serão apenas analisados os resultados do cenário 1 para evitar duplicação de pensamento e conclusões.

7.3 Recursos Utilizados

Por fim, são apresentadas tabelas que mostram os recursos utilizados por cada sistema de cache. Todos os valores são normalizados em relação à API base para facilitar a comparação entre as configurações. No entanto, a API base também inclui valores absolutos, permitindo assim a conversão direta entre medidas normalizadas e reais.

A primeira tabela refere-se ao esforço computacional associado à API. A segunda tabela segue o mesmo princípio de normalização, mas diz respeito especificamente ao esforço computacional na base de dados.

Tabela 7.1: Resultados de performance para a API.

Cache	Avg CPU	Max CPU	Avg RAM	Max RAM
Base	1.8 %	68.3 %	3.80 GB	6.40 GB
Caffeine	0.889	0.676	1.526	1.047
Ehcache	3.556	0.836	1.447	1.172
Guava	0.778	0.717	1.211	0.906
Ignite*	7.611	2.097	1.487	2.029

Nota: Os valores estão normalizados em relação á linha **Base** (valor / Base). A linha **Base** mostra os valores brutos.
* O Ignite inclui uma componente extra responsável pelo processamento distribuído. Todos os valores de CPU e memoria estão considerados como consumo da API.

Tabela 7.2: Resultados de performance para a base de dados.

Cache	Avg CPU	Max CPU	Avg RAM	Max RAM
Base	95.2 %	107.8 %	3.14 GB	3.22 GB
Caffeine	1.047	1.019	1.003	1.000
Ehcache	1.022	1.020	0.981	1.016
Guava	1.054	1.026	1.013	1.003
Ignite	0.996	1.007	0.987	0.997

Nota: Os valores estão normalizados em relação á linha **Base** (valor / Base). A linha **Base** mostra os valores brutos.

7.4 Análise dos Resultados

7.4.1 Métrica ICE

A métrica **ICE** (Indicador de eficiência de cache) mede como uma cache melhora ou degrada o tempo de execução e a taxa de acertos em relação a uma linha de base, usando contribuições ponderadas.

A métrica é definida como:

$$\text{ICE} = \left(k_3 \cdot T_{\text{imp}}\right) + \left(k_4 \cdot H_{\text{imp}}\right) - \left(k_1 \cdot T_{\text{deg}}\right) - \left(k_2 \cdot H_{\text{deg}}\right) \tag{7.1}$$

onde:

$$\begin{aligned}
T_{\text{imp}} &= \max(0, -T_d), && \text{(melhora no tempo \%)} \\
T_{\text{deg}} &= \max(0, T_d), && \text{(degradação no tempo \%)} \\
H_{\text{imp}} &= \max(0, H_d), && \text{(melhora na taxa de acertos \%)} \\
H_{\text{deg}} &= \max(0, -H_d), && \text{(degradação na taxa de acertos \%)} \\
k_1, k_2, k_3, k_4 &\geq 0, && \text{(pesos relativos)}
\end{aligned}$$

Esta fórmula é altamente flexível, permitindo que desenvolvedores priorizem ou penalizem parâmetros específicos de acordo com os requisitos do sistema. Isso permite enfatizar métricas relacionadas a erros, enquanto fatores como latência podem ser reduzidos quando apropriado.

Entretanto, um valor alto de ICE não indica necessariamente que um cache é universalmente melhor que outra. Ele apenas reflete um melhor desempenho sob as condições específicas e os pesos escolhidos para aquele cenário. Portanto, os desenvolvedores devem selecionar os pesos cuidadosamente antes de realizar os cálculos, garantindo que reflitam as prioridades operacionais do sistema. Além disso, a métrica ICE não considera o consumo de recursos ou outras características distintas dos mecanismos de cache, que devem ser avaliadas separadamente.

7.4.2 Pesos Utilizados

Nesta secção, usamos a mesma fórmula para calcular o ICE, usando o conjunto de três pesos diferentes a seguir, para cada uma das databases (Light e Heavy)e do cenário 1 (portátil):

$$\text{Weights a)}(1 \cdot T_{\text{imp}}) + (2 \cdot H_{\text{imp}}) - (1.1 \cdot T_{\text{deg}}) - (2.2 \cdot H_{\text{deg}})$$

$$\text{Weights b)}(2 \cdot T_{\text{imp}}) + (1 \cdot H_{\text{imp}}) - (2.2 \cdot T_{\text{deg}}) - (1.5 \cdot H_{\text{deg}})$$

$$\text{Weights c)}(1 \cdot T_{\text{imp}}) + (1 \cdot H_{\text{imp}}) - (1 \cdot T_{\text{deg}}) - (1 \cdot H_{\text{deg}})$$

O primeiro conjunto de pesos concentra-se em melhorar a qualidade da resposta, no qual as respostas corretas são mais valorizadas e os erros são penalizados. O fator tempo é recompensado, mas tem um peso limitado quando o desempenho melhora e um peso ligeiramente maior quando o desempenho piora. O segundo conjunto de ponderações é o oposto: prioriza respostas rápidas e penaliza as mais lentas enquanto penaliza moderadamente as respostas incorretas. Por fim, o ultimo conjunto pondera todos os valores por igual simulando um sistema que procura de forma igual respostas corretas e rápidas.

7.4.3 Heavy Database

7.4.3.1 Cenários de baixa concorrência

Começando a analise com o caso de baixa concorrência, foi calculado todos os valores de ICE para o conjunto de pesos acima. Todos os valores correspondentes são apresentados na Tabela7.3.

Tabela 7.3: Valores ICE para cenários de baixa concorrência.

Cache	Modo	a	b	c
Caffeine	Cold	9.18	11.34	6.84
Caffeine	Operational	5.80	8.63	4.63
Ehcache	Cold	2.29	-5.93	-2.45
Ehcache	Operational	4.24	1.79	2.03
Guava	Cold	7.36	11.03	6.13
Guava	Operational	8.16	10.99	6.39
Ignite	Cold	14.56	-10.65	-5.87
Ignite	Operational	1.40	48.35	20.4

Estes valores permitem agora criar um ranking de performance para cada conjunto de pesos.

$$\text{Guava} > \text{Caffeine} > \text{Ehcache} > \text{Ignite} \tag{7.2}$$

$$\text{Ignite} > \text{Guava} > \text{Caffeine} > \text{Ehcache} \tag{7.3}$$

$$\text{Ignite} > \text{Guava} > \text{Caffeine} > \text{Ehcache} \tag{7.4}$$

7.4.3.2 Cenários de média concorrência

Seguindo o mesmo raciocínio no contexto de média concorrência. Todos os valores correspondentes são apresentados na Tabela7.4.

Tabela 7.4: Valores ICE para cenários de média concorrência.

Cache	Modo	a	b	c
Caffeine	Cold	-11.74	4.07	-0.34
Caffeine	Operational	8.14	22.08	10.54
Ehcache	Cold	10.95	-7.81	1.85
Ehcache	Operational	3.47	-4.5	0.07
Guava	Cold	38.3	7.6	16
Guava	Operational	23.64	9.23	11.12
Ignite	Cold	6.46	-17.08	-2.31
Ignite	Operational	24.48	65.63	36.84

Estes valores permitem agora criar um ranking para cada conjunto de pesos.

$$\text{Ignite} > \text{Guava} > \text{Caffeine} > \text{Ehcache} \tag{7.5}$$

$$\text{Ignite} > \text{Caffeine} > \text{Guava} > \text{Ehcache} \tag{7.6}$$

$$\text{Ignite} > \text{Guava} > \text{Caffeine} > \text{Ehcache} \tag{7.7}$$

7.4.3.3 Cenários de alta concorrência

Finalmente, no nível de alta concorrência, o Ehcache domina com o maior valor da tabela (25.64), superando o Ignite, que atinge 21.27. Outra observação relevante é que todos os caches exceto o Caffeine mostram desempenho aprimorado nesta métrica durante a execução operacional quando comparados à execução cold.

Todos os valores correspondentes são apresentados na Tabela7.5.

Tabela 7.5: Valores ICE para cenários de alta concorrência.

Cache	Modo	a	b	c
Caffeine	Cold	3.66	0.68	2.34
Caffeine	Operational	5.44	0.83	6.05
Ehcache	Cold	31.12	19.94	17.02
Ehcache	Operational	48.44	27.58	25.34
Guava	Cold	40.84	4.63	23.35
Guava	Operational	57.77	6.83	32.97
Ignite	Cold	7.61	5.94	5.61
Ignite	Operational	25.17	38.64	21.27

Estes valores permitem agora criar um ranking para cada conjunto de pesos.

$$\text{Guava} > \text{Ehcache} > \text{Ignite} > \text{Caffeine} \tag{7.8}$$
$$\text{Ignite} > \text{Ehcache} > \text{Guava} > \text{Caffeine} \tag{7.9}$$
$$\text{Guava} > \text{Ehcache} > \text{Ignite} > \text{Caffeine} \tag{7.10}$$

7.4.4 Light Database

7.4.4.1 Cenários de baixa concorrência

Começando a analise com o caso de baixa concorrência, foi calculado todos os valores de ICE para o conjunto de pesos acima. Todos os valores correspondentes são apresentados na Tabela7.6.

Tabela 7.6: Valores ICE para cenários de baixa concorrência.

Cache	Modo	a	b	c
Caffeine	Cold	2.48	4.96	2.48
Caffeine	Operational	55.38	110.76	55.38
Ehcache	Cold	0.82	1.64	0.82
Ehcache	Operational	63.63	127.26	63.63
Guava	Cold	4.95	9.9	4.95
Guava	Operational	5.78	11.56	5.78
Ignite	Cold	-2.04	-0.31	-0.48
Ignite	Operational	53.69	131.35	63.6

Estes valores permitem agora criar um ranking de performance para cada conjunto de pesos.

$$\text{Ehcache} > \text{Caffeine} > \text{Ignite} > \text{Guava} \tag{7.11}$$
$$\text{Ignite} > \text{Ehcache} > \text{Caffeine} > \text{Guava} \tag{7.12}$$
$$\text{Ignite} > \text{Ehcache} > \text{Caffeine} > \text{Guava} \tag{7.13}$$

7.4.4.2 Cenários de média concorrência

Seguindo o mesmo raciocínio no contexto de média concorrência.

Todos os valores correspondentes são apresentados na Tabela7.7.

Tabela 7.7: Valores ICE para cenários de média concorrência.

Cache	Modo	a	b	c
Caffeine	Cold	0.84	1.68	0.84
Caffeine	Operational	55.08	110.16	55.08
Ehcache	Cold	-0.84	-1.84	-0.84
Ehcache	Operational	63.55	127.1	63.55
Guava	Cold	0	0	0
Guava	Operational	2.54	5.08	2.54
Ignite	Cold	-2.86	-1.95	-1.3
Ignite	Operational	48.68	121.43	58.64

Estes valores permitem agora criar um ranking para cada conjunto de pesos.

$$\text{Ehcache} > \text{Caffeine} > \text{Ignite} > \text{Guava} \tag{7.14}$$

$$\text{Ehcache} > \text{Ignite} > \text{Caffeine} > \text{Guava} \tag{7.15}$$

$$\text{Ehcache} > \text{Ignite} > \text{Caffeine} > \text{Guava} \tag{7.16}$$

7.4.4.3 Cenários de alta concorrência

Finalmente, no nível de alta concorrência, o Ehcache domina com o maior valor da tabela (25.64), superando o Ignite, que atinge 21.27. Outra observação relevante é que todos os caches exceto o Caffeine mostram desempenho aprimorado nesta métrica durante a execução operacional quando comparados à execução cold.

Todos os valores correspondentes são apresentados na Tabela7.8.

Tabela 7.8: Valores ICE para cenários de alta concorrência.

Cache	Modo	a	b	c
Caffeine	Cold	0	0	0
Caffeine	Operational	53.84	107.6	53.84
Ehcache	Cold	0	0	0
Ehcache	Operational	62.39	124.78	62.39
Guava	Cold	0.85	1.7	0.85
Guava	Operational	3.41	6.82	3.41
Ignite	Cold	-3.79	-3.82	-2.15
Ignite	Operational	51.82	127.71	61.78

Estes valores permitem agora criar um ranking para cada conjunto de pesos.

$$\text{Ehcache} > \text{Caffeine} > \text{Ignite} > \text{Guava} \tag{7.17}$$

$$\text{Ignite} > \text{Ehcache} > \text{Caffeine} > \text{Guava} \tag{7.18}$$

$$\text{Ehcache} > \text{Ignite} > \text{Caffeine} > \text{Guava} \tag{7.19}$$

7.4.5 Analise dos Recursos

A análise detalhada dos resultados de consumo de recursos revela diferenças significativas entre os sistemas de cache avaliados. Na camada da API, os valores normalizados mostram que o Ignite possui um consumo médio de CPU 7,6 vezes maior que a API base, enquanto os caches Caffeine e Guava mantêm consumo abaixo da base, em 0.89 e 0.78, respectivamente. O Ehcache apresenta um valor intermediário (3.56), indicando um esforço considerável, mas ainda inferior ao Ignite. O consumo máximo de CPU do Ignite também é maior, atingindo 2.1 vezes o valor da base, enquanto os demais caches permanecem abaixo de 1, o que mostra que o Ignite impõe um alto custo ao CPU devido ao processamento distribuído e seus componentes.

Em termos de memória, todos as caches apresentam economia em relação à API base, exceto o Ignite, cujo uso máximo de RAM chega a 2.03 vezes o valor da base, refletindo o overhead adicional necessário para manter consistência e replicação de dados em ambientes distribuídos. Guava e Caffeine são novamente os mais econômicos, com uso máximo de RAM de 0.91 e 1.05 vezes a base, respectivamente, enquanto o Ehcache permanece próximo, porém com maior consumo (1.17).

Ao analisar o impacto no banco de dados, observa-se que o Ignite é a único cache que consegue reduzir ligeiramente o consumo médio de CPU da base de dados para 0.996 da base, e também mostra uma pequena redução no uso médio de RAM (0.987), sugerindo que, apesar do seu alto custo na API, ele efetivamente contribui para aliviar a carga na base de dados. Caffeine e Guava, por outro lado, mostram um aumento marginal no consumo de CPU da base de dados (1.047 e 1.054), indicando que, embora eficientes na API, o seu efeito na redução da carga na base de dados é menos significativo. O Ehcache mantém valores próximos de 1 para CPU e RAM, refletindo um comportamento intermediário, com impacto neutro na base de dados.

Capítulo

8

Conclusões

8.1 Introdução

Ao longo desta tese foram abordados diversos aspetos relacionados com o conceito de cache, desde a origem do termo e a sua necessidade, até às tecnologias existentes e às principais características que definem os diferentes mecanismos de cache. Foi identificada uma lacuna na literatura e apresentado um possível contributo para o seu preenchimento. Foi igualmente apresentada a metodologia adotada no estudo, bem como a motivação subjacente, descrevendo-se de forma detalhada a implementação realizada e os resultados obtidos. Adicionalmente, foi proposta uma equação para a avaliação de soluções de cache, cuja aplicabilidade prática foi demonstrada através dos cenários analisados.

Neste capítulo final, são sintetizados os principais resultados alcançados e discutidas as limitações do estudo, bem como possíveis direções para trabalhos futuros.

8.2 Resultados

Com base nos valores observados no capítulo anterior, é possível concluir que não existe uma resposta universal à questão de qual a melhor solução de cache. Pelo contrário, a escolha da solução mais adequada depende fortemente do sistema em análise, dos objetivos pretendidos e do nível de carga a que este está sujeito. Para além disso, é fundamental considerar as características específicas e inerentes a cada solução estudada, bem como as suas limitações e compromissos. Estes fatores reforçam a ideia de que não existe uma solução única que seja superior em todos os contextos.

Ainda assim, esta tese apresenta diferentes cenários de teste, desde níveis de concorrência a base de dados, permitindo uma análise comparativa mais abrangente. A métrica proposta surge como uma contribuição interessante, uma vez que pode ser utilizada para avaliar soluções de cache em diferentes sistemas, ficando ao critério do gestor do projeto a definição dos pesos que melhor se adequem às necessidades e prioridades do seu contexto específico.

Outra contribuição é a possível solução para o sistema de cache para o projeto EPOS onde devido as caraterísticas do mesmo seria recomendado o Ehcache pela sua persistência de dados ou ignorando esse ponto bastante favorável a implementação de Caffeine como sendo a que mais se sobressai dentro destas. O Ignite é descartado devido a elevada complexidade falta de mecanismos de remoção e atualização e o processamento distribuído neste projeto ser demasiado complexo para implementar em todos os parceitros do projeto EPOS.

8.3 Trabalho Futuro

Conforme referido anteriormente, esta tese propõe uma métrica destinada a apoiar a análise e a seleção de soluções de cache. No entanto, a sua aplicabilidade não se restringe a esse objetivo, podendo também ser utilizada para avaliar a eficácia de diferentes algoritmos de evicção ou para comparar o desempenho de caches com base nas suas estratégias de alocação de dados (heap e non-heap). Adicionalmente, a métrica pode ser refinada através da introdução de pesos associados aos recursos utilizados e a características relevantes, como a persistência de dados, de forma a refletir a sua importância relativa. Nesse sentido, um trabalho futuro relevante consistiria no estudo e na calibração desses pesos, com vista à definição de um conjunto de pesos globais mais representativo e ajustável a diferentes contextos de utilização.

Uma das principais limitações deste trabalho prende-se com a impossibilidade de testar as soluções em um maior número de ambientes distintos, com diferentes capacidades e recursos de hardware reais. A realização de testes adicionais em contextos mais diversificados permitiria tornar o estudo mais abrangente e aumentar a generalização dos resultados obtidos.

Como trabalho futuro, sugere-se a expansão da análise para outros cenários e arquiteturas, bem como o refinamento da métrica proposta através da análise do conjunto de dados e da definição de pesos mais adequados para diferentes situações. Este processo poderá recorrer a técnicas de aprendizagem automática, com o objetivo de aumentar a robustez da métrica e a sua aplicabilidade prática.

Adicionalmente, a realização de testes que considerem diferentes latências de rede permitiria evidenciar de forma mais clara os benefícios da utilização de cache, nomeadamente em termos de tempo de resposta, quando comparada com cenários em que esta não é empregue. A avaliação em ambientes com diferentes condições de rede, bem como em múltiplos cenários e arquiteturas, contribuiria ainda para uma maior generalização e validação dos resultados obtidos.

Bibliografia

[1] GeeksforGeeks, "Difference between two-tier and three-tier database architecture," Disponível em: https://www.geeksforgeeks.org/difference-between-two-tier-and-three-tier-database-architecture/, 2025, acesso em: fev. 2025.

[2] G. C. Stierhoff and A. G. Davis, "A history of the ibm systems journal," *IEEE Annals of the History of Computing*, vol. 20, no. 1, pp. 29–35, 1998.

[3] D. Jiang, "Performance guarantees for web applications," Ph.D. dissertation, Vrije Universiteit Amsterdam, 2012.

[4] IBM, "Three-tier architecture: Guia sobre arquitetura em três camadas," Disponível em: https://www.ibm.com/br-pt/topics/three-tier-architecture, 2025, acesso em: jan. 2025.

[5] S. Sivasubramanian, G. Pierre, M. van Steen, and G. Alonso, "Analysis of caching and replication strategies for web applications," *IEEE Internet Computing*, vol. 11, no. 1, pp. 60–66, 2007.

[6] C. Mohan, "Caching technologies for web applications," Disponível em: http://www.almaden.ibm.com/u/mohan/Caching_VLDB2001.pdf, 2001, iBM Almaden Research Center.

[7] J. Mertz and I. Nunes, "A qualitative study of application-level caching," *IEEE Transactions on Software Engineering*, vol. 43, no. 9, pp. 798–816, 2017.

[8] M. W. Docs, "Etag – http," Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Headers/ETag, 2025, acesso em: jan. 2025.

[9] R. Fielding and J. Reschke, "Hypertext transfer protocol (http/1.1): Conditional requests," Disponível em: https://www.rfc-editor.org/rfc/rfc7232, 2014.

[10] P. U. C. do Rio Grande do Sul, "Integridade de dados em memórias cache," Disponível em: https://www.inf.pucrs.br/emoreno/

undergraduate/CC/orgarqii/class_files/Aula14a.pdf, 2018, acesso em: jan. 2025.

[11] U. F. de Pernambuco, "Memórias cache: Uma introdução," Disponível em: https://cin.ufpe.br/~can/Arquivos/cache.notes-2.pdf, 2006, acesso em: jan. 2025.

[12] A. T. Kabakuş, "A performance comparison of java cache memory implementations," *Gümüşhane Üniversitesi Fen Bilimleri Enstitüsü Dergisi*, vol. 10, no. 3, pp. 844–852, 2020.

[13] G. Hasslinger, M. Okhovatzadeh, K. Ntougias, F. Hasslinger, and O. Hohlfeld, "An overview of analysis methods and evaluation results for caching strategies," *arXiv*, 2023.

[14] D. Noetzold, A. G. M. Rossetto, L. A. Silva, P. Crocker, and V. R. Q. Leithardt, "Jvm optimization: An empirical analysis of jvm configurations for enhanced web application performance," *SoftwareX*, vol. 28, p. 101933, 2024.

[15] S. Kanthed, "Redis vs. memcached in microservices architectures: Caching strategies," *International Journal of Multidisciplinary Research and Growth Evaluation*, vol. 4, no. 3, pp. 1084–1091, 2023.

[16] I. Henckel and D. Söderberg, "Database loading strategies for an in-memory cache in java," Master's thesis, Lund University, 2022.

[17] M. I. Zulfa, R. Hartanto, and A. E. Permanasari, "Caching strategy for web application – a systematic literature review," *International Journal of Web Information Systems*, vol. 16, no. 5, pp. 545–569, 2020.

[18] Google, "Guava: Google core libraries for java," Disponível em: https://github.com/google/guava, 2025.

[19] A. S. Foundation, "Apache ignite: In-memory computing platform," Disponível em: https://ignite.apache.org/, 2025.

[20] S. AG, "Ehcache: Open source, standards-based cache," Disponível em: https://www.ehcache.org/, 2025.

[21] Cache4J, "Cache4j: A simple and flexible caching library for java," Disponível em: https://github.com/ehcache/ehcache, 2025.

[22] B. Manes, "Caffeine: A high performance caching library," Disponível em: https://github.com/ben-manes/caffeine, 2025.

[23] Payara, "Developing web applications," Disponível em: https://docs.payara.fish/enterprise/docs/Technical%20Documentation/Application%20Development/Developing%20Web%20Applications.html, 2025, acesso em: fev. 2025.

[24] R. Labs, "Redis: The real-time data platform," Disponível em: https://redis.io/, 2025, acesso em: ago. 2025.

[25] D. Mehrotra, R. Nagpal, and P. K. Bhatia, "Designing metrics for caching techniques for dynamic web site," *International Conference on Computer and Communication Technology (ICCCT)*, pp. 589–595, 2010.

[26] A. W. Services, "Implementing microservices on aws," Disponível em: https://docs.aws.amazon.com/pdfs/whitepapers/latest/microservices-on-aws/microservices-on-aws.pdf, 2025.

[27] V. Senra, "Estudos e implementações sobre caching em java," Disponível em: https://github.com/vms2100/Tese, 2025, repositório GitHub.